Plug-in CaWaQS-Viz 0003.58

Visualiseur pour la plate-forme de modélisation CaWaQS 3.57

Guide d'utilisation

Nicolas FLIPO

Simone MAZZARELLI

Nicolas GALLOIS

Lise-Marie GIROD

10 novembre 2026

Mines Paris/ARMINES, PSL Université - Centre de Géosciences
35, rue Saint-Honoré
F-77 305 FONTAINEBLEAU Cedex

Table des matières

Introduction **vi**

Nouveautés **vii**

I Guide utilisateur **1**

1 Pré-requis et installation **2**
1.1 Option 1 — environnement Pixi (recommandée) . . . 3
1.1.1 Installer `pixi` (une fois par machine) . . . 3
1.1.2 Cloner le dépôt, initialiser le backend et installer l'environnement . . . 3
1.1.3 Lancer QGIS . . . 4
1.1.4 Mettre à jour le plug-in . . . 4
1.2 Option 2 — gestionnaire d'extensions QGIS via ZIP . . . 4
1.2.1 Construire un ZIP d'installation propre . . . 5
1.2.2 Installer le ZIP dans QGIS . . . 6
1.2.3 Ce qui se passe au premier lancement . . . 7
1.3 Paramètres d'installation avancés (DEV) . . . 8
1.3.1 Installation des dépendances . . . 8
1.3.2 Installation pour les développeurs . . . 8

2 Configuration **10**
2.1 Généralités . . . 11
2.1.1 Mode de configuration de CaWaQS-Viz . . . 12
2.2 Détection automatique des paramètres du projet . . . 13
2.2.1 Sonde 1 — depuis le répertoire de sorties CaWaQS . . . 13
2.2.2 Sonde 2 — depuis les voisins du projet QGIS . . . 14
2.3 Configuration du projet à partir d'un fichier `.json` . . . 15

3 Fonctionnalités de CaWaQS-Viz **16**
3.1 "`Calibration Tools`" : outils de calage . . . 17
3.1.1 WIP : modification des fichiers de paramétrisation . . . 18
3.1.2 WIP : lier une table attributaire à sa résolution . . . 19
3.1.3 `Statistical Criteria` : calcul des critères de performance statistique aux points d'observation . . . 20
3.1.3.1 Critères statistiques calculés par la boîte de dialogue "`Statistical criteria`" 20
3.1.3.2 Sortie de la boîte de dialogue "`Statistical Criteria`" . . . 21
3.1.4 WIP `Run-off coefficient calculation` : calcul du coefficient de ruissellement aux stations de mesure de débit . . . 22
3.2 `Compare sim/obs` : comparaison des chroniques observées et simulées . . . 24
3.3 `Spatial Map` : cartographie spatiale des variables hydrologiques . . . 25
3.4 `Budget Bar Plot` : visualisation du bilan hydrologique . . . 26
3.5 `Hydological Regime` : visualisation du régime hydrologique . . . 27
3.6 `Extract Data from a sub area` : extraction des résultats dans un polygone . . . 28
3.6.1 Paramètres . . . 28
3.6.2 Sorties . . . 28
3.6.3 Unités . . . 29
3.6.4 Paramètres — valeurs internes . . . 29
3.6.4.1 Le GeoPackage des valeurs internes . . . 29
3.6.5 Paramètres — valeurs aux limites . . . 30
3.6.5.1 Convention d'unité et de signe . . . 30
3.6.5.2 Mode par défaut : le CSV des flux de faces . . . 30
3.6.5.3 Mode GeoPackage : le GeoPackage des limites AQ . . . 30
3.6.6 Utilisation de l'extraction depuis un notebook . . . 31
3.6.6.1 Valeurs internes : les tuples `specs` . . . 31
3.6.6.2 Valeurs aux limites : `mask_aq_boundary` . . . 33

4 Utiliser CaWaQS-Viz depuis un notebook (sans QGIS) **34**
4.1 Utiliser CaWaQS-Viz depuis un notebook (sans QGIS) - pré-requis . . . 35
4.2 Mettre en place l'environnement . . . 35
4.3 Exécuter sur un serveur distant . . . 35
4.4 Choisir un modèle . . . 36
4.5 Flux de travail . . . 36

5 Configuration manuelle du projet (avancé) **37**
5.1 Contenu du projet QGIS utilisé pour le traitement des résultats de simulation CaWaQS . . . 38
5.2 Configurer la géométrie du projet depuis un projet QGIS existant . . . 40
5.3 Configurer les résolutions depuis un fichier de configuration `.json` . . . 44

II Guide du développeur **46**

6 Concepts clés pour les développeurs **47**
6.1 Rattacher les points d'observation aux mailles . . . 48
6.2 Le compartiment HYD : `ID_ABS` contre `ID_GIS` . . . 48

7 Architecture **50**
7.1 Architecture séparée : HTAS et CWV . . . 51
7.2 Les trois couches du backend . . . 51
7.3 Exemple de flux de travail : le Budget Bar Plot . . . 54
7.4 Portes d'entrée . . . 55
7.4.1 Porte client — L1 - HT CLIENT - MACRO . . . 55
7.4.2 Porte développeur — L2 - HT DEVELOPER - MICRO . . . 55

8 Flux de développement **57**
8.1 Ajouter une nouvelle opération de dialogue . . . 57
8.2 Lors d'une modification — liste de contrôle . . . 57
8.3 Conventions . . . 58
8.4 Mise à jour de version . . . 58
8.5 Documentation du code . . . 59

9 Outillage du développeur **61**
9.1 Outils de débogage . . . 62
9.1.1 L'extension `Plugin Reloader` . . . 62
9.1.2 L'extension QGIS `First Aid` . . . 62
9.1.3 VS-Code . . . 63
9.2 Créer des boîtes de dialogue avec `Qt Designer` . . . 67

Bibliographie **68**

Introduction

L'outil de visualisation CaWaQS-Viz a été développé dans le but de visualiser les sorties de modélisation de la plate-forme de modélisation `CaWaQS` 3.57. Cette extension fonctionne actuellement comme un plug-in QGIS sous la version 3.34de QGIS, et les mêmes fonctionnalités sont également accessibles par API à travers un notebook Jupyter .

CaWaQS-Viz s'articule autour de différents outils et fonctionnalités qui accompagnent le processus de calage d'une application CaWaQS et facilitent la visualisation des chroniques temporelles et de la distribution spatiale des variables hydrologiques simulées.

CaWaQS-Viz s'appuie sur un backend scientifique autonome, HTAS (HydrologicalTwinAlphaSeries), qu'il consomme au travers d'une interface publique bien définie.

La Fig. 1 situe le plug-in au sein de la chaîne d'outils CaWaQS.

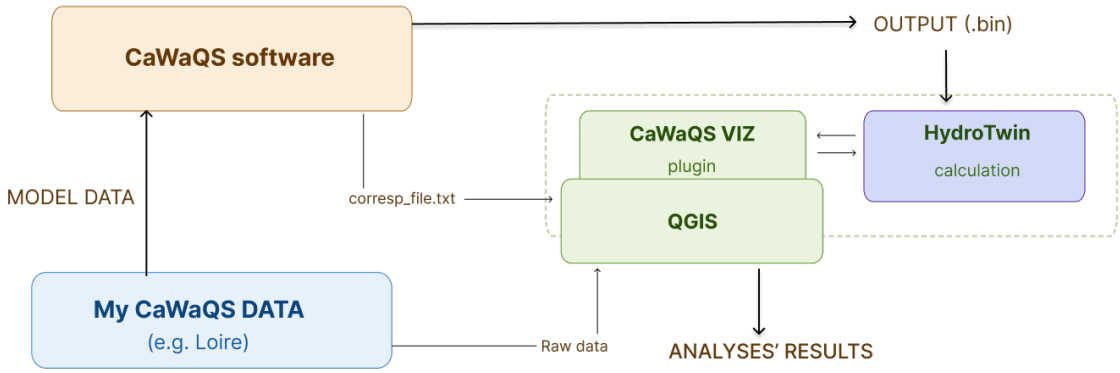


Figure 1 – L'écosystème CaWaQS-Viz — position du plug-in au sein de la chaîne d'outils CaWaQS.

Nouveautés

Voici la liste des derniers correctifs et des dernières implémentations de fonctionnalité dans l'application CaWaQS-Viz.

Type/Tag	Description	Section
Nouvelle fonction	Séparation du backend et du frontend dans deux dépôts distincts reliés par des API. Le backend constitue la première version du Jumeau Hydrologique : Hydrological Twin Alpha Series (HTAS)	Sec. 2.1
Nouvelle fonction	Automatisation de la configuration : seul le répertoire de sorties CaWaQS est nécessaire	Sec. 5.2
Nouvelle fonction	Extraction par masque. Découpage des résultats sur une sous-zone à partir d'un shapefile	Sec. 3.6

Tableau 1 – Dernières implémentations dans l'application

Derniers tags stables :

Tag CWV_alpha_3.72 : Lien (05 juin 2026) : tag stable de CaWaQS-Viz couplé à HTAS.

Première partie

Guide utilisateur

CHAPITRE 1

Pré-requis et installation

Sommaire

1.1 Option 1 — environnement Pixi (recommandée) . . . 3
1.1.1 Installer `pixi` (une fois par machine) . . . 3
1.1.2 Cloner le dépôt, initialiser le backend et installer l'environnement . . . 3
1.1.3 Lancer QGIS . . . 4
1.1.4 Mettre à jour le plug-in . . . 4
1.2 Option 2 — gestionnaire d'extensions QGIS via ZIP . . . 4
1.2.1 Construire un ZIP d'installation propre . . . 5
1.2.2 Installer le ZIP dans QGIS . . . 6
1.2.3 Ce qui se passe au premier lancement . . . 7
1.3 Paramètres d'installation avancés (DEV) . . . 8
1.3.1 Installation des dépendances . . . 8
1.3.2 Installation pour les développeurs . . . 8

Le plug-in CaWaQS-Viz a été conçu pour fonctionner sous le système d'information géographique (SIG) QGIS. Il a initialement été développé sous QGIS 3.28 [1], et est actuellement utilisé sous sa version 3.36. Son bon fonctionnement n'est pas garanti pour une utilisation sous une version ultérieure.

CaWaQS-Viz s'appuie sur un backend scientifique, `HydrologicalTwinAlphaSeries` (HTAS), embarqué dans le dépôt comme sous-module GitLab sous `external/HydrologicalTwinAlphaSeries`. Ce sous-module doit être renseigné pour que le plug-in puisse fonctionner, quelle que soit l'option d'installation retenue.

Deux options d'installation sont disponibles :

- **Option 1 — environnement Pixi** (Sec. 1.1) : installe QGIS et l'ensemble des dépendances à l'intérieur du dossier du projet, de manière reproductible et autonome. Recommandée pour les développeurs et pour des déploiements reproductibles.
- **Option 2 — gestionnaire d'extensions QGIS via ZIP** (Sec. 1.2) : l'archive de plug-in classique, installée depuis QGIS. Plus pratique pour les utilisateurs finaux, mais plus fragile pour la gestion des dépendances.

1.1 Option 1 — environnement Pixi (recommandée)

Cette option installe QGIS et l'ensemble des dépendances à l'intérieur du dossier du projet *via* `pixi`. Elle fournit un environnement reproductible et autonome dans lequel QGIS, Python et chaque dépendance sont figés.

1.1.1 Installer `pixi` (une fois par machine)

Voici les informations reprises de la page d'installation de Pixi :
Sous macOS ou Linux :

```
curl -fsSL https://pixi.sh/install.sh | sh
source $HOME/.bashrc
```

Si votre système ne dispose pas de `curl`, utilisez `wget` :

```
wget -qO- https://pixi.sh/install.sh | sh
```

Sous Windows :

```
powershell -ExecutionPolicy Bypass -c "irm -useb https://pixi.sh/install.ps1 | iex"
```

1.1.2 Cloner le dépôt, initialiser le backend et installer l'environnement

1. Disponible à l'adresse : https://www.qgis.org/fr/site/forusers/download.html

```
mkdir $HOME/.plugins-qgis
cd $HOME/.plugins-qgis
git clone https://gitlab.com/gviz/cawaqsviz.git
cd cawaqsviz
git submodule update --init external/HydrologicalTwinAlphaSeries
pixi install
```

Depuis la racine du dépôt **cawaqsviz**, initialisez uniquement **external/HydrologicalTwinAlphaSeries**. N'utilisez pas de mise à jour récursive des sous-modules, sauf si vous avez explicitement besoin des sources de la documentation interne du backend et que vous avez accès à leurs dépôts privés.

1.1.3 Lancer QGIS

```
pixi run cawaqsviz
```

La dernière étape consiste à activer le plug-in dans QGIS depuis le menu principal : "**Extensions**" → "**Installer/Gérer les extensions**", puis à sélectionner "**Installées**" dans la fenêtre de navigation qui s'ouvre (Fig. 1.2).

1.1.4 Mettre à jour le plug-in

Une fois l'installation effectuée, placez-vous dans le répertoire **cawaqsviz**, mettez-le éventuellement à jour vers la dernière version, puis lancez-le avec **pixi** :

```
cd $HOME/.plugins-qgis/cawaqsviz
git pull                                                    # met a jour cawaqsviz ;
    peut deplacer le SHA fige du backend
git submodule sync -- external/HydrologicalTwinAlphaSeries  # rafraichit l'URL
    distante du backend si elle a change en amont
git submodule update --init external/HydrologicalTwinAlphaSeries  # place le backend sur
    le SHA fige par cawaqsviz
pixi run cawaqsviz                                          # lance QGIS depuis l'
    environnement pixi
```

1.2 Option 2 — gestionnaire d'extensions QGIS via ZIP

L'outil de visualisation CaWaQS-Viz a été développé et testé sous QGIS 3.28 et versions ultérieures. ATTENTION : ce type d'installation est plus fragile que l'Option 1 pour la gestion des dépendances. L'équipe de développement recommande l'Option 1 pour le développement à partir des sources et pour des environnements reproductibles.

> N'utilisez **pas** le bouton "**Download ZIP**" de GitLab — son archive casse l'installation du plug-in. Vous devez construire le ZIP vous-même à partir d'un **git clone** correct incluant le contenu des sous-modules, comme indiqué ci-dessous.

Le téléchargement de l'archive GitLab échoue pour deux raisons :

- le dossier de premier niveau porte le nom de la branche ou du tag (par ex. **cawaqsviz-main-abc1234/**), alors que QGIS exige qu'il soit nommé exactement **cawaqsviz**. L'installation directe du ZIP GitLab produit une erreur du type **No module named 'cawaqsviz-<branch>'**;
- le contenu des sous-modules GitLab n'est pas inclus — seulement un pointeur. Le dossier embarqué **external/HydrologicalTwinAlphaSeries/** sera vide et l'amorçage au premier lancement échouera avec une erreur de type « sous-module manquant ».

1.2.1 Construire un ZIP d'installation propre

Sous Linux/macOS :

```
# 1. Cloner dans un emplacement temporaire avec le contenu des sous-modules
cd /tmp
git clone https://gitlab.com/gviz/cawaqsviz.git
cd cawaqsviz
git submodule update --init external/HydrologicalTwinAlphaSeries

# 2. Construire le ZIP depuis le repertoire parent, en excluant (facultatif) les artefacts
    propres au developpement
cd ..
zip -r cawaqsviz.zip cawaqsviz \
    -x "cawaqsviz/.git/*" \
    -x "cawaqsviz/.pixi/*" \
    -x "cawaqsviz/__pycache__/*" \
    -x "cawaqsviz/**/__pycache__/*"
```

Sous Windows (PowerShell), utilisez Git for Windows pour cloner (les mêmes commandes **git** que ci-dessus), puis construisez le ZIP en cliquant droit sur le dossier **cawaqsviz/** (→ "**Envoyer vers**" → "**Dossier compressé**"), ou avec PowerShell :

```
Compress-Archive -Path cawaqsviz -DestinationPath cawaqsviz.zip
```

Le **cawaqsviz.zip** obtenu porte le bon nom de dossier de premier niveau et inclut le contenu du sous-module backend.

1.2.2 Installer le ZIP dans QGIS

Dans la barre supérieure de menus de QGIS, cliquez sur "**`Extensions`**" puis "**`Installer/Gérer les extensions`**". La fenêtre de gestion et d'import des extensions s'ouvre. Sélectionnez "**`Installer depuis un ZIP`**" (Fig. 1.1) et choisissez **`cawaqsviz.zip`**, puis cliquez sur "**`Installer l'extension`**".

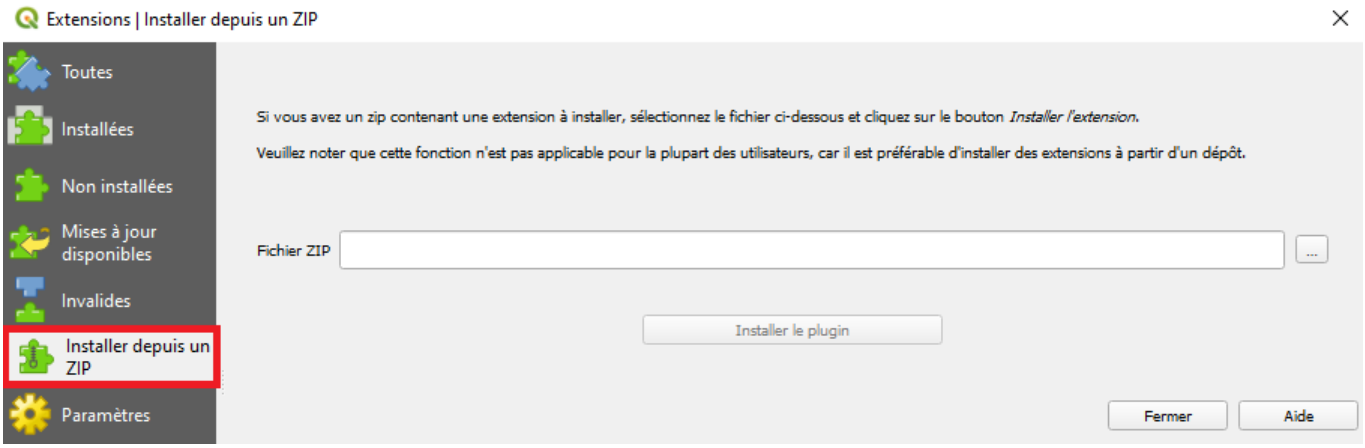


Figure 1.1 – Boîte de dialogue QGIS pour la gestion d'extensions.

Une fois le fichier **`.zip`** décompressé sous QGIS, il peut être nécessaire d'activer ce dernier dans la même fenêtre de gestion des extensions en naviguant dans le volet "**`Installées`**". L'extension CaWaQS-Viz doit être cochée pour figurer dans la barre d'outils QGIS (Fig. 1.2).

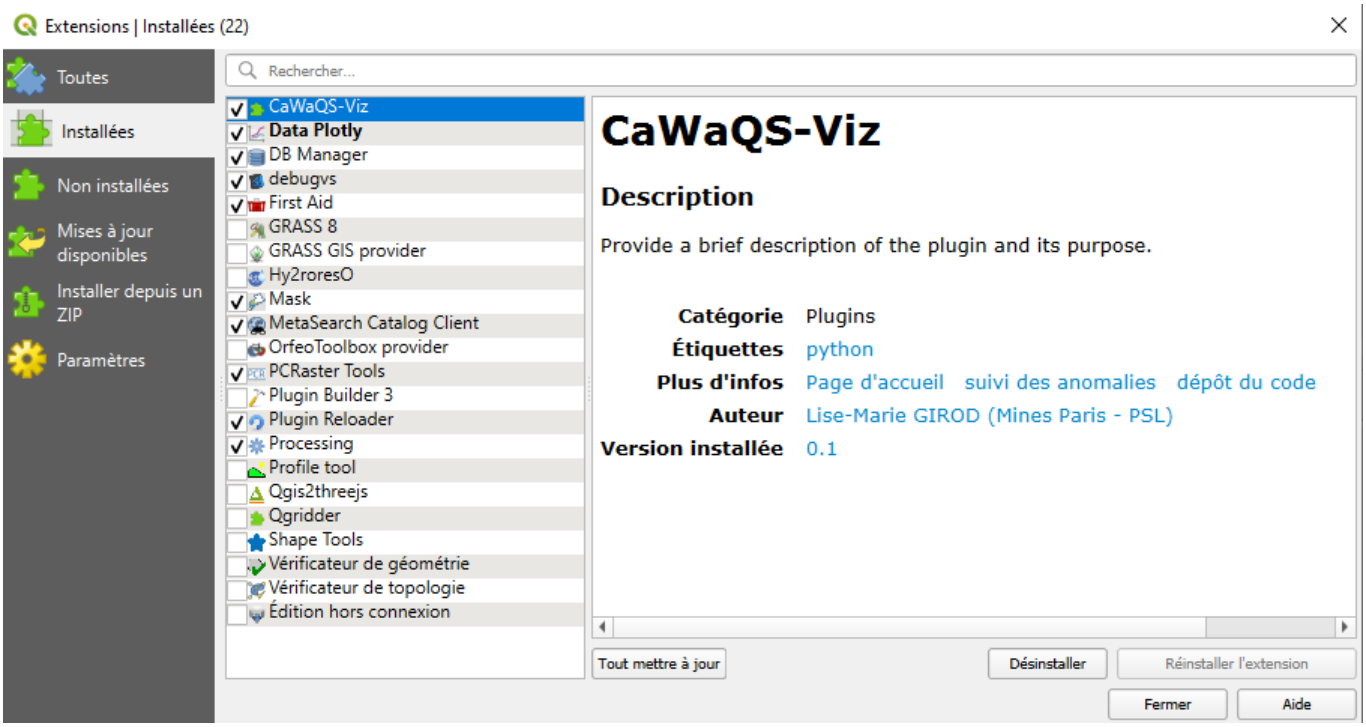


Figure 1.2 – Activation de l'extension CaWaQS-Viz dans l'application QGIS.

Une fois l'application présente dans la barre d'outils de QGIS, celle-ci est prête à être lancée d'un simple clic sur l'icône .

1.2.3 Ce qui se passe au premier lancement

Au premier lancement après une installation par ZIP, le plug-in exécute automatiquement `pip install -user -upgrade external/HydrologicalTwinAlphaSeries` pour enregistrer le backend et résoudre ses dépendances Python. Cela nécessite une connexion internet et s'installe dans les site-packages propres à votre utilisateur (`~/.local/lib/pythonX.Y/site-packages` sous Linux, `%APPDATA%\Python\...` sous Windows). Les lancements suivants sont sans effet.

Sous Debian/Ubuntu et dérivés qui embarquent un Python système « externally-managed » (PEP 668), le plug-in retente automatiquement avec `-break-system-packages` (l'installation atterrit tout de même dans vos site-packages utilisateur, et non dans les chemins système gérés par apt).

Si l'installation échoue, la console Python de QGIS affiche la commande `pip` exacte que vous pouvez exécuter manuellement pour diagnostiquer.

> Pour utiliser toute nouvelle mise à jour du plug-in installé via ZIP, il sera nécessaire de réitérer l'intégralité de cette procédure (Sec. 1.2). Avec l'Option 1, la mise à jour se fait plutôt avec `git pull` (Sec. 1.1.4).

1.3 Paramètres d'installation avancés (DEV)

1.3.1 Installation des dépendances

Les dépendances sont gérées automatiquement. Avec l'Option 1, **pixi** fige et installe QGIS, Python et chaque dépendance à l'intérieur du dossier du projet (Sec. 1.1). Avec l'Option 2, l'amorçage au premier lancement résout le backend et ses dépendances Python (Sec. 1.2.3). Aucune installation manuelle de dépendances n'est normalement nécessaire.



Si l'amorçage au premier lancement échoue, la console Python de QGIS affiche la commande **pip** exacte à exécuter. En dernier recours, les dépendances peuvent tout de même être installées manuellement depuis l'interpréteur Python intégré à QGIS. Les installer depuis un autre interpréteur ne permettra pas à QGIS d'accéder aux librairies nécessaires.

Les principales dépendances Python sont : **pandas**, **numpy**, **ipython**, **matplotlib** et **geopandas**. En solution de repli manuelle, ouvrez une console Python de QGIS [2] et exécutez :

```
>>> import pip
>>> pip.main(['install', 'pandas', 'numpy', 'ipython', 'matplotlib', 'geopandas'])
```

1.3.2 Installation pour les développeurs

À des fins de développement, et pour faciliter la mise à jour du plug-in depuis l'interface QGIS, le plug-in peut être installé directement dans le répertoire des extensions *natives* de QGIS. Le dépôt GitLab de CaWaQS-Viz peut être cloné depuis un terminal de commande, en pensant à initialiser le sous-module backend :

```
$ git clone https://gitlab.com/gviz/cawaqsviz
$ cd cawaqsviz
$ git submodule update --init external/HydrologicalTwinAlphaSeries
```

Le répertoire de l'application doit être placé à l'emplacement (dossier) des applications natives de QGIS afin que toute modification du code source du plug-in puisse être prise en compte par QGIS lors de son démarrage. Le répertoire QGIS est accessible *via* le chemin suivant :

```
#windows
C:/Programmes/QGIS3.X.X.X/apps/qgis-ltr/python/plugins/
#linux
~/.local/share/QGIS/QGIS3/profiles/default/python/plugins/
```

2. En cliquant sur dans l'interface QGIS.

La structure du code du plug-in est décrite au chapitre 7.



L'autocomplétion des fonctionnalités liées à la librairie qgis est disponible à condition d'utiliser l'interpréteur Python employé par QGIS. Un environnement virtuel peut être initialisé à cet effet.

N.B. : Pour faciliter le développement du plug-in, les utilitaires suivants peuvent être installés depuis le gestionnaire des applications QGIS :

1. **Reloader** permet de recharger le plug-in sans relancer QGIS à chaque modification de code,
2. **FirstAid** permet une visualisation détaillée des erreurs retournées par l'interpréteur Python de QGIS ainsi que des différents objets et variables liés à l'extension.

CHAPITRE 2

Configuration

Sommaire

2.1 Généralités **11**
2.1.1 Mode de configuration de CaWaQS-Viz 12
2.2 Détection automatique des paramètres du projet **13**
2.2.1 Sonde 1 — depuis le répertoire de sorties CaWaQS 13
2.2.2 Sonde 2 — depuis les voisins du projet QGIS 14
2.3 Configuration du projet à partir d'un fichier `.json` **15**

2.1 Généralités

L'extension CaWaQS-Viz intègre plusieurs fonctionnalités accessibles depuis sa boîte de dialogue d'accueil (Fig. 2.1) :

- une entrée dédiée à la configuration de l'application CaWaQS ("`Settings`" - Sec. 2.1.1 à 2.3) ;
- des outils d'aide au calage ("`Calibration tools`" - Sec. 3.1) ;
- des fonctionnalités de visualisation des chroniques temporelles de débits et de piézométrie simulées et observées ("`Compare Sim/Obs`" - Sec. 3.2) ;
- des outils de visualisation de la distribution spatiale des composantes hydrologiques ("`Spatial Maps`" - Sec. 3.3) ;
- une fonction de visualisation du bilan hydrologique ("`Budget Bar Plot`" - Sec. 3.4) ;
- un visualiseur de régime hydrologique ("`Hydrological regime`" - Sec. 3.5).

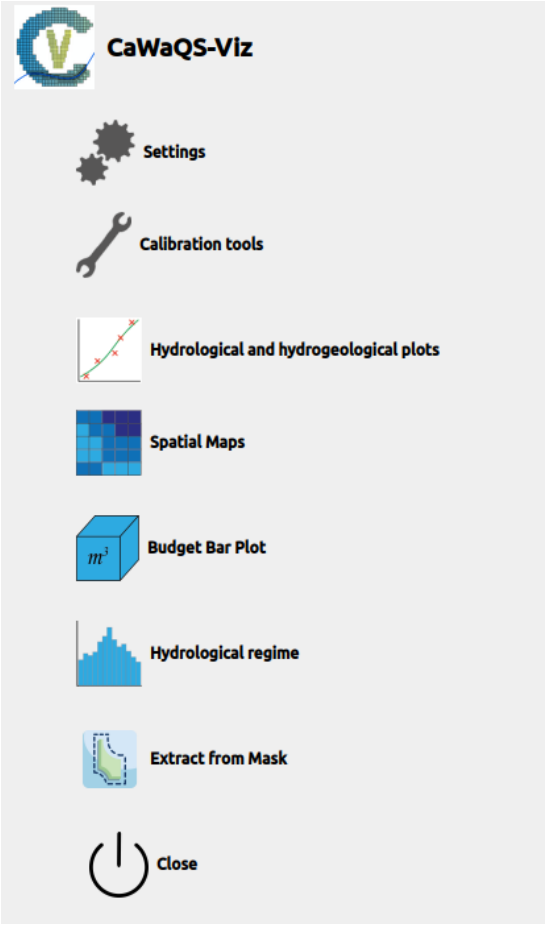


Figure 2.1 – Boîte de dialogue d'accueil de l'application CaWaQS-Viz.

L'utilisation de l'extension CaWaQS-Viz repose sur un projet `SIG` préexistant contenant :

- les maillages des différents modules de l'application CaWaQS, aux résolutions de sortie de la modélisation ;

- des points d'observation et/ou des points d'extraction, situés à l'intérieur des limites de l'emprise du modèle.

2.1.1 Mode de configuration de CaWaQS-Viz

Le plug-in doit être configuré pour lire les différents fichiers de sortie de la modélisation et/ou les fichiers d'entrée de CaWaQS ainsi que les différents maillages. Ce paramétrage du plug-in peut être réalisé dans la fenêtre "`Settings`". Pour ce faire, deux procédures sont disponibles 2.2 :

- PREMIÈRE FOIS : configurer le projet manuellement dans la seconde partie de la boîte de dialogue (section 2.2).
- DÉJÀ CONFIGURÉ : importer un fichier de configuration `.json` établi au préalable (section 2.3).

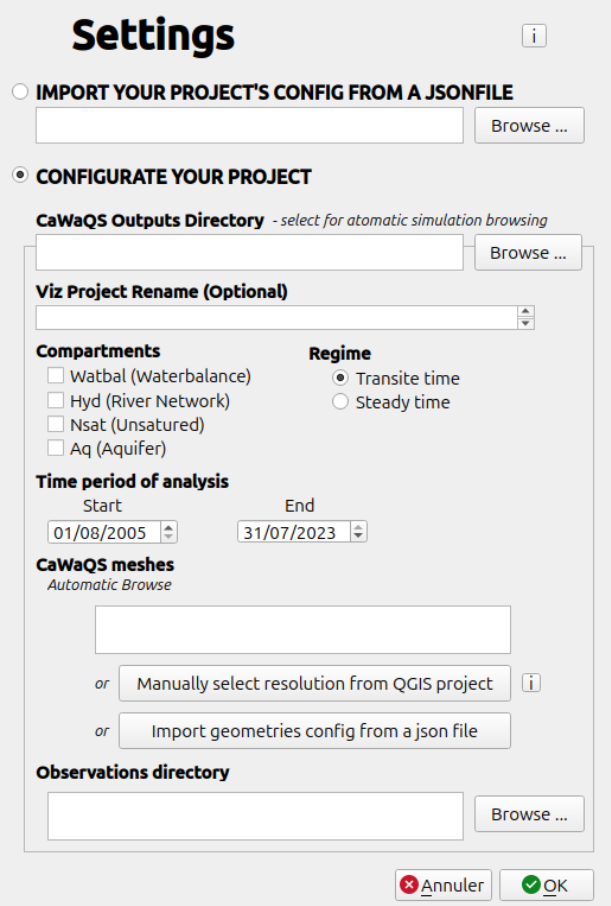


Figure 2.2 – La boîte de dialogue "`Settings`" du plug-in, avec la création d'un nouveau projet sélectionnée.

Le détail de ces deux procédures de configuration de l'application est décrit respectivement aux paragraphes 2.2 et 2.3.

2.2 Détection automatique des paramètres du projet

Lors de la configuration manuelle d'un projet (plutôt que par import d'un fichier de configuration `.json`), CaWaQS-Viz peut pré-remplir la majeure partie de la boîte de dialogue "`Settings`" en inspectant le système de fichiers. La seule action requise de l'utilisateur est de parcourir jusqu'au répertoire de sorties CaWaQS : la détection s'exécute alors automatiquement et renseigne les compartiments, les années de simulation, le régime, le répertoire d'observations et le chemin de configuration des géométries.

La détection est un mécanisme de *meilleur effort*, à visée de commodité. Elle repose entièrement sur les conventions de nommage et d'organisation décrites ci-dessous. Si une convention n'est pas respectée, le champ correspondant est simplement laissé vide et peut être renseigné à la main — la seule exigence stricte est que le répertoire de sorties contienne au moins un sous-dossier de module reconnaissable (voir Sec. 2.2.1).

La détection s'appuie sur deux sondes indépendantes, ancrées sur deux racines différentes :

- le **répertoire de sorties** CaWaQS que vous sélectionnez (Sec. 2.2.1) — pilote les compartiments, les années et le régime ;
- le **fichier de projet QGIS** (`.qgz` / `.qgs`) actuellement ouvert (Sec. 2.2.2) — pilote la configuration des géométries, le répertoire d'observations et le nom du projet.

2.2.1 Sonde 1 — depuis le répertoire de sorties CaWaQS

Cette sonde est ancrée sur le répertoire de sorties sélectionné via le bouton "`Browse`". Elle attend, directement dans ce répertoire, un ou plusieurs sous-dossiers de sortie de module, chacun nommé d'après son module CaWaQS :

Sous-dossier	Compartiment détecté
`Output_AQ`	`AQ` (aquifères)
`Output_HYD`	`HYD` (réseau hydraulique)
`Output_WATBAL`	`WATBAL` (bilan hydrique de surface)
`Output_NONSAT`	`NSAT` (zone non saturée)

Au moins un de ces sous-dossiers doit être présent ; sinon la détection échoue et la boîte de dialogue doit être renseignée manuellement.

À l'intérieur de chaque sous-dossier de module, les **noms des fichiers de résultats binaires** déterminent les années de simulation et le régime. Deux motifs de nommage sont reconnus :

- `<MODULE>_<VAR>.YYYYYYYY.bin` — un pas de temps annuel **transitoire**. Les huit chiffres encodent deux années consécutives, l'année de début `YYYY` suivie de l'année de fin `YYYY`, la fin valant le début $+1$ (par ex. `20102011`). Chaque fichier de ce type contribue par son année de début ; le début le plus ancien et la fin la plus récente définissent la plage de simulation.
- `<MODULE>_<VAR>.00.bin` — un résultat en régime **permanent**. La présence de ce motif signale une simulation permanente.

Le régime est résolu comme suit : `Steady` si seuls des noms de fichiers permanents sont trouvés et aucun transitoire ; sinon `Transient`. Si les deux motifs coexistent, la simulation est classée `Transient` et un avertissement est émis. Si aucune année n'a pu être lue dans un nom de fichier, la plage vaut par défaut `1970–2022`.

L'organisation attendue ressemble donc à ceci :

```
<OUT_CAW>/
|-- Output_AQ/
|     |-- AQ_HEAD.20102011.bin      <-- transitoire (2010->2011)
|     |-- AQ_HEAD.20112012.bin      <-- transitoire (2011->2012)
|     +-- ...
|-- Output_HYD/
|     +-- HYD_Q.20102011.bin
|-- Output_WATBAL/
|     +-- WATBAL_...00.bin          <-- marqueur de regime permanent
+-- Output_NONSAT/
      +-- ...
```

Code 2.1 – Organisation attendue du répertoire de sorties CaWaQS.



Le régime, la liste des compartiments et les années de simulation sont lus *uniquement* par cette sonde. Parcourir jusqu'au répertoire de sorties est donc l'unique étape obligatoire pour la détection automatique.

2.2.2 Sonde 2 — depuis les voisins du projet QGIS

Cette sonde est ancrée sur le fichier de projet QGIS actuellement ouvert (`.qgz` / `.qgs`). Elle relève purement du meilleur effort : chacun des trois éléments ci-dessous est recherché indépendamment, et n'importe lequel peut rester vide sans empêcher la détection.

- **Configuration des géométries** — la sonde recherche les fichiers correspondant à `config_geometries_*.json` dans le *même dossier* que le fichier de projet. Si plusieurs correspondent, le plus récemment modifié est retenu.
- **Répertoire d'observations** — la sonde cherche un dossier nommé `DATA_OBS` au niveau du fichier de projet, puis un niveau au-dessus, puis deux niveaux au-dessus, et s'arrête à la première correspondance.
- **Nom du projet** — déduit du dossier qui *contient* `DATA_OBS` lorsqu'il est trouvé ; sinon du nom du dossier parent (ou grand-parent) du fichier de projet.

Le voisinage attendu du fichier de projet est le suivant :

```
<project_root>/                     <-- nom utilise comme nom de projet
|-- DATA_OBS/                       <-- repertoire d'observations
+-- <qgis_folder>/
      |-- my_project.qgz            <-- projet QGIS ouvert
      +-- config_geometries_*.json  <-- config des geometries (le plus recent l'emporte)
```

Code 2.2 – Organisation attendue autour du fichier de projet QGIS.



Comme la configuration des géométries n'est recherchée qu'à côté du fichier `.qgz` / `.qgs`, et `DATA_OBS` seulement jusqu'à deux niveaux au-dessus, un projet enregistré en dehors de sa structure de dossiers habituelle laissera ces champs vides. C'est le comportement attendu — renseignez-les manuellement via le bouton "`Configure Geometry`" et le champ de parcours des observations.

2.3 Configuration du projet à partir d'un fichier `.json`

Une fois un projet configuré (que ce soit par détection automatique ou à la main), sa configuration est écrite sous forme de fichiers `.json` dans `<OUT_CAW>/PostProcessing/CONFIG/`, à savoir `config_project_<name>.json` et `config_geometries_<name>.json`. Ces fichiers peuvent ensuite être ré-importés directement via la première procédure de configuration, contournant entièrement la détection.

L'import d'un fichier `.json` externe peut aussi servir à configurer le projet. Ce fichier est automatiquement enregistré dans le répertoire de post-traitement d'un projet après sa première configuration.

À l'exécution de la boîte de dialogue de paramétrage de l'application CaWaQS-Viz, le sous-répertoire du projet est créé dans le répertoire de post-traitement spécifié. Si le projet n'existe pas encore dans ce répertoire, un sous-répertoire portant le nom du projet est créé, contenant les fichiers de configuration du projet et des résolutions écrits au format "`json`".

Ce fichier `.json` de configuration de projet est structuré comme suit :

```
{
    "json_path_geometries": str,
    "projectName": str,
    "cawOutDirectory": str,
    "startSim": int,
    "endSim": int,
    "obsDirectory": str,
    "ppDirectory": str,
    "regime": str
}
```

Le contenu de chaque objet de ce fichier `.json` est le suivant :

- "`json_path_geometries`" : chemin absolu du fichier de configuration des résolutions ;
- "`projectName`" : nom du projet ;
- "`cawOutDirectory`" : chemin absolu du répertoire contenant les sorties de modélisation CaWaQS ;
- "`startSim`" et "`endSim`" : années de début et de fin de simulation au format `integer` ;
- "`obsDirectory`" : chemin absolu du répertoire parent contenant les chroniques observées ;
- "`ppDirectory`" : chemin absolu du répertoire de post-traitement. Notez que ce chemin doit déjà exister. Dans le cas contraire, il ne sera pas créé par l'application.
- "`regime`" : régime de modélisation (« Transient » ou « Steady »).

CHAPITRE 3

Fonctionnalités de CaWaQS-Viz

Sommaire

3.1 "Calibration Tools" : outils de calage 17
3.1.1 WIP : modification des fichiers de paramétrisation 18
3.1.2 WIP : lier une table attributaire à sa résolution 19
3.1.3 `Statistical Criteria` : calcul des critères de performance statistique aux points d'observation 20
3.1.4 WIP `Run-off coefficient calculation` : calcul du coefficient de ruissellement aux stations de mesure de débit 22
3.2 `Compare sim/obs` : comparaison des chroniques observées et simulées 24
3.3 `Spatial Map` : cartographie spatiale des variables hydrologiques 25
3.4 `Budget Bar Plot` : visualisation du bilan hydrologique 26
3.5 `Hydological Regime` : visualisation du régime hydrologique 27
3.6 `Extract Data from a sub area` : extraction des résultats dans un polygone 28
3.6.1 Paramètres 28
3.6.2 Sorties 28
3.6.3 Unités 29
3.6.4 Paramètres — valeurs internes 29
3.6.5 Paramètres — valeurs aux limites 30
3.6.6 Utilisation de l'extraction depuis un notebook 31

3.1 "Calibration Tools" : outils de calage

La boîte de dialogue "**Calibration Tools**" (Fig. 3.1) propose des outils utiles pour caler un nouveau modèle CaWaQS. Ces fonctionnalités ne sont pas toutes opérationnelles à ce jour. En effet, la majorité d'entre elles n'a pas été prise en compte lors de la refonte du T1 2026. Elles restent néanmoins déjà présentes dans le code. Une restructuration et un raccordement minimes suffiraient à les rendre à nouveau opérationnelles.

> Toutes les fonctionnalités non encore opérationnelles sont signalées dans le guide utilisateur par l'acronyme WIP (Work in Progress) ou par un symbole /!\ dans QGIS.

- "**Modify a parametrisation file**" (Sec. 3.1.1) - WIP : outil de modification des fichiers de paramétrisation des différents compartiments modélisés ;
- "**Statistical Criteria**" (Sec. 3.1.3) : outil de calcul des critères de performance du calage du modèle ;
- "**Run-off coefficient calculation**" (Sec. 3.1.4) WIP : outil de calcul des coefficients de ruissellement simulés ;
- "**Compare sim/obs**" (Sec. 3.2) WIP : outil de visualisation des chroniques simulées et observées.

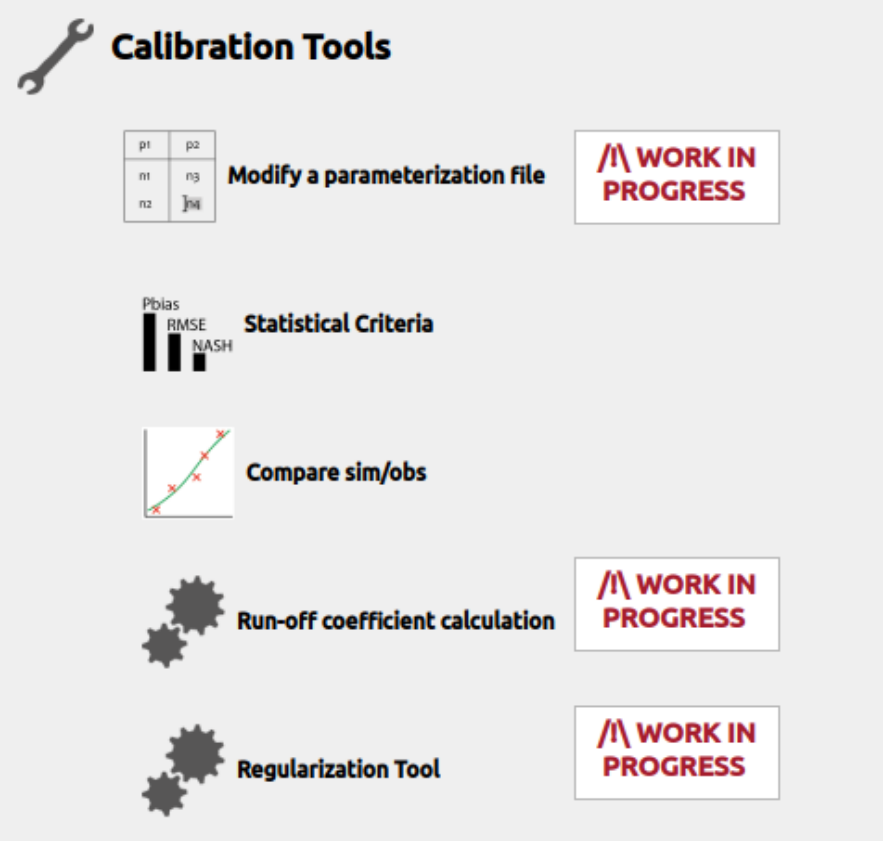
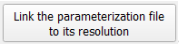


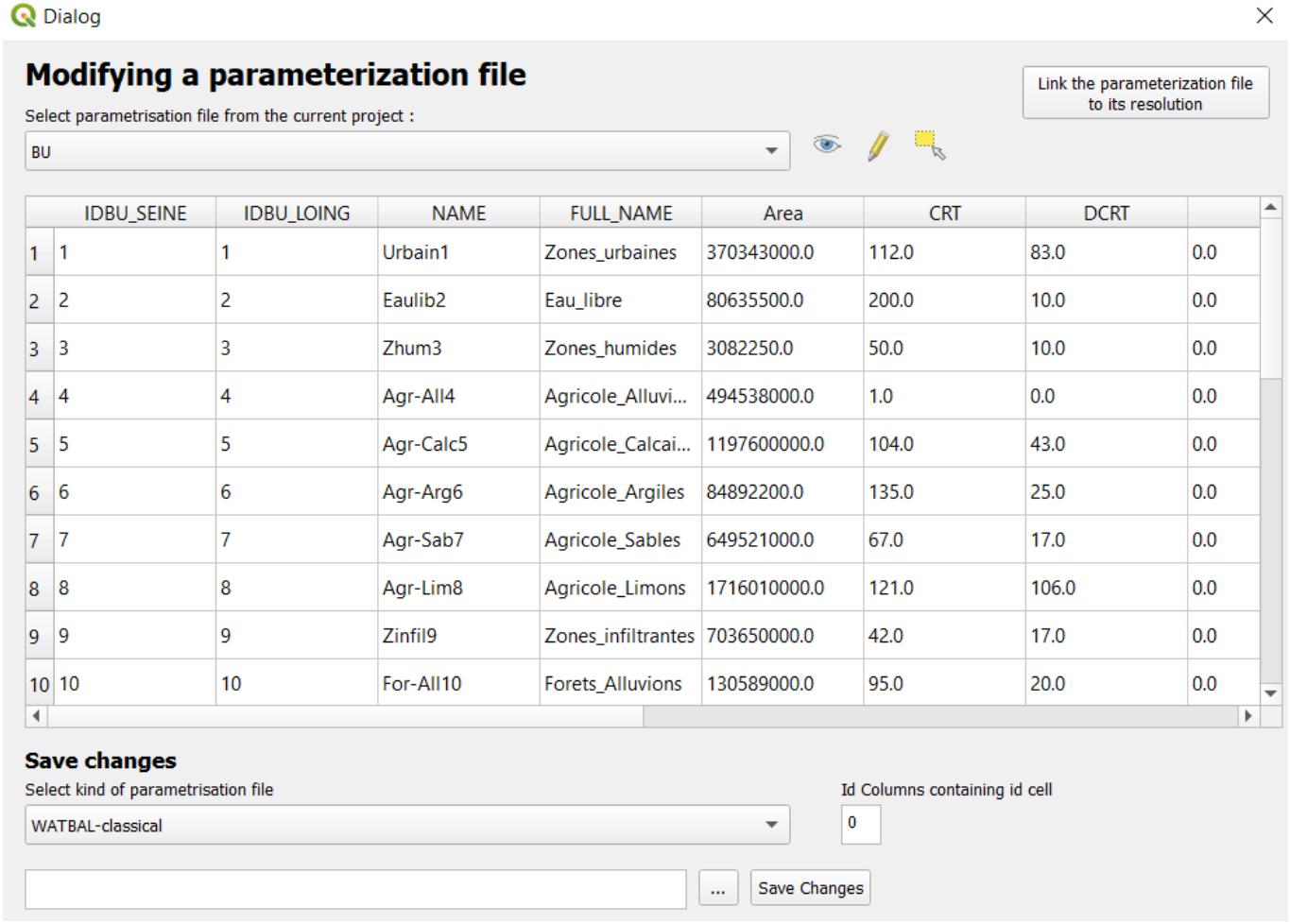
Figure 3.1 – Boîte de dialogue des outils de calage.

3.1.1 WIP : modification des fichiers de paramétrisation

Les fichiers de paramétrisation peuvent être modifiés depuis la boîte de dialogue "`Modifying a parametrization file`" (Fig. 3.2). La résolution dépendra du fichier de calage à sélectionner dans le menu déroulant. La table peut être affichée en cliquant sur  .

Si la table attributaire affichée ne contient pas les champs du fichier de paramétrisation, la résolution peut être liée à son fichier de paramétrisation dans la boîte de dialogue "`Link resolution to its parameterisation file`", accessible en cliquant sur 

. L'utilisation de cette boîte de dialogue est décrite en Sec. 3.1.2.



	IDBU_SEINE	IDBU_LOING	NAME	FULL_NAME	Area	CRT	DCRT	
1	1	1	Urbain1	Zones_urbaines	370343000.0	112.0	83.0	0.0
2	2	2	Eaulib2	Eau_libre	80635500.0	200.0	10.0	0.0
3	3	3	Zhum3	Zones_humides	3082250.0	50.0	10.0	0.0
4	4	4	Agr-All4	Agricole_Alluvi...	494538000.0	1.0	0.0	0.0
5	5	5	Agr-Calc5	Agricole_Calcai...	1197600000.0	104.0	43.0	0.0
6	6	6	Agr-Arg6	Agricole_Argiles	84892200.0	135.0	25.0	0.0
7	7	7	Agr-Sab7	Agricole_Sables	649521000.0	67.0	17.0	0.0
8	8	8	Agr-Lim8	Agricole_Limons	1716010000.0	121.0	106.0	0.0
9	9	9	Zinfil9	Zones_infiltrantes	703650000.0	42.0	17.0	0.0
10	10	10	For-All10	Forets_Alluvions	130589000.0	95.0	20.0	0.0

Figure 3.2 – Boîte de dialogue de modification des fichiers de paramétrisation.

Une fois les paramètres modifiés dans la table affichée, celle-ci peut être enregistrée. Le nouveau chemin du fichier peut être spécifié en sélectionnant 

. Précisez l'identifiant de la colonne contenant l'identifiant ou le nom de la maille [1].

1. ATTENTION : pour rappel, les identifiants de colonnes commencent à **0**.

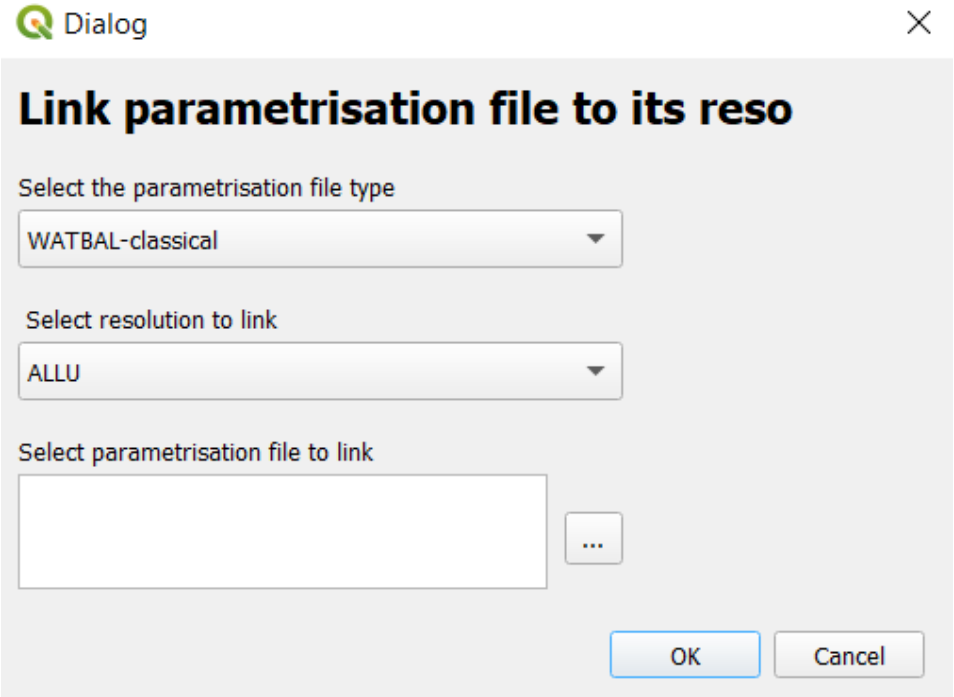


Figure 3.3 – Boîte de dialogue de liaison d'une table attributaire à sa résolution.

3.1.2 WIP : lier une table attributaire à sa résolution

Les paramètres du fichier de paramétrisation seront directement importés comme champs dans la table attributaire du fichier de résolution concerné. Pour créer ce lien, il faut choisir un type de fichier de paramétrisation (selon le compartiment et le type de fichier de paramétrisation), la résolution CaWaQS, et le chemin vers ce fichier de paramétrisation. Différents types de fichiers de paramétrisation existent et sont listés dans la table 3.1.

Compartiment	Type de fichier de paramétrisation	Paramètres listés dans la table
`WATBAL`	Classique	CRT, DCRT, R, FN, CQR, CQRMAX, CQI, CQIMAX et RNAP
`WATBAL`	Excès d'infiltration	CRT, DCRT, R, FN, CQR, CQRMAX, CQI, CQIMAX, RNAP et RINF
`AQ`	Transmissivité	T
`AQ`	Emmagasinement (coefficient d'emmagasinement)	S
`AQ`	Conductance	C

Tableau 3.1 – Fichiers de paramétrisation pris en charge par `CaWaQS-Viz`.

3.1.3 `Statistical Criteria` : calcul des critères de performance statistique aux points d'observation

La boîte de dialogue "**`Statistical Criteria`**" (Fig. 3.4) renvoie une couche SIG suivant les géométries de la résolution d'observation propre à la variable analysée. Chaque champ correspond à un critère statistique donné. Les critères statistiques sélectionnés sont calculés sur la période spécifiée dans la section "**`Period for calculating statistical criteria`**".

3.1.3.1 Critères statistiques calculés par la boîte de dialogue "`Statistical criteria`"

"`Pbias`" : biais [%]

$$PBIAS = \frac{\sum(\hat{y}_i - y_i)}{\sum y_i}.100 \qquad \begin{array}{ll} y_i & \text{: variable simulée au temps } i \\ O_i & \text{: variable observée au temps } i \end{array}$$

"`Average SIM/OBS`"

$$A = \overline{\hat{y}} - \overline{y} \qquad \begin{array}{ll} \overline{\hat{y}} : & \text{moyenne des variables simulées} \\ \overline{y} : & \text{moyenne des variables observées} \end{array}$$

"`Nash`" : efficience de Nash-Sutcliffe (Nash et Sutcliffe, 1970)

$$NSE = 1 - \frac{\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{n}(y_i - \overline{y})^2} \qquad \begin{array}{ll} y_i : & \text{variable réelle au temps } i \\ \hat{y}_i : & \text{variable prédite au temps } i \end{array}$$

"RMSE" - erreur quadratique moyenne

$$RMSE = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2} \quad \text{où} \quad \begin{array}{ll} n : & \text{nombre total de jours de simulation} \\ y_i : & \text{variable réelle au temps } i \\ \hat{y}_i : & \text{variable prédite au temps } i \end{array}$$

"KGE" - efficience de Kling-Gupta (Gupta *et al.*, 2012)

$$KGE = 1 - \sqrt{(r-1)^2 + (\alpha - 1)^2 + (\beta - 1)^2} \quad \begin{array}{ll} r & : \text{coefficient de corrélation de Pearson entre les séries de variables simulées et observées} \\ \alpha & : \text{rapport des écarts-types simulé et observé} \\ \beta & : \text{biais moyen entre séries simulée et observée} \end{array} \tag{3.1}$$

Les critères suivants sont calculés automatiquement : $\overline{y}$, $\overline{\hat{y}}$, $\frac{\overline{\hat{y}}}{\overline{y}}$, σy, $\sigma \hat{y}$, $\frac{\sigma \hat{y}}{\sigma y}$.

3.1.3.2 Sortie de la boîte de dialogue "Statistical Criteria"

La boîte de dialogue génère des critères statistiques :

- aux points d'observation,
- à l'échelle de la couche aquifère dans le cas d'une simulation avec compartiment aquifère ;
- à l'échelle de l'hydrosystème dans le cas d'une simulation avec compartiment aquifère.

Critères statistiques calculés aux points d'observation

La boîte de dialogue génère la table des critères statistiques aux points d'observation aux formats **shapefile** et **texte**. Le shapefile est directement intégré au projet **QGIS** courant, tandis que le fichier texte se trouvera dans le répertoire **../PostProcessing/STATS_CRITS**. Les critères en sortie sont contenus dans les champs de chaque point de mesure sous les noms définis dans la table 3.2.

Critères statistiques calculés à l'échelle des unités aquifères

Dans le cas d'un calcul des critères statistiques sur le compartiment aquifère, un fichier **texte** est enregistré dans le répertoire "**Post-Processing**" sous **../PostProcessing/STATS_CRITS/AQ_crits_xxxx_xxxx.txt**. Ce fichier liste les critères statistiques de la table 3.2 définis à l'échelle de chaque unité aquifère.

Critères statistiques calculés à l'échelle de l'hydrosystème

Des critères statistiques sont définis à l'échelle de l'hydrosystème pour la charge hydraulique souterraine. Ces critères sont écrits dans le fichier **../PostProcessing/STATS_CRITS/AQ_crits_xxxx_xxxx.txt**.

Toutes les sorties shapefile générées par CaWaQS-Viz sont temporaires. Si elles ne sont pas enregistrées, elles ne seront plus accessibles à la réouverture du projet.

Nom du champ de la table de sortie	Critère statistique
"NObs"	Nombre d'observations sur la période sélectionnée
"AObs"	Moyenne des variables observées
"ASIM"	Moyenne des variables simulées
"SUM_RATIO"	Rapport de la somme des variables simulées à la somme des variables observées
"StdObs"	Écart-type des variables observées
"StdSim"	Écart-type des variables simulées
"STD_RATION"	Rapport des écarts-types des variables observées et simulées
"PBAIS"	Biais [%]
"AVERAGE SIM/OBS"	Rapport moyen entre variables simulées et observées
"KGE"	Efficience de Kling-Gupta
"NASH"	Efficience de Nash-Sutcliffe
"RMSE"	Erreur quadratique moyenne

Tableau 3.2 – Noms des champs des tables de sortie pour chaque critère statistique

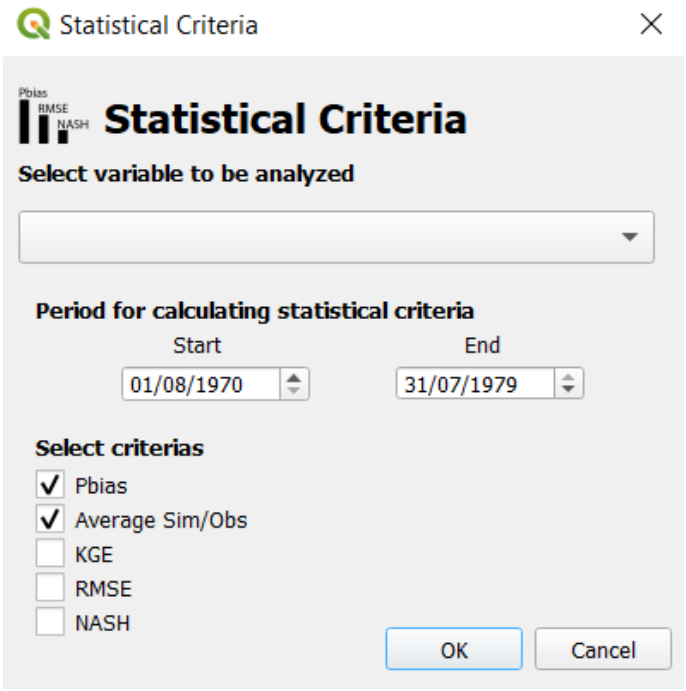


Figure 3.4 – Boîte de dialogue "Statistical Criteria".

3.1.4 WIP Run-off coefficient calculation : calcul du coefficient de ruissellement aux stations de mesure de débit

Cette boîte de dialogue calcule le coefficient de ruissellement simulé et observé aux stations de mesure de débit.

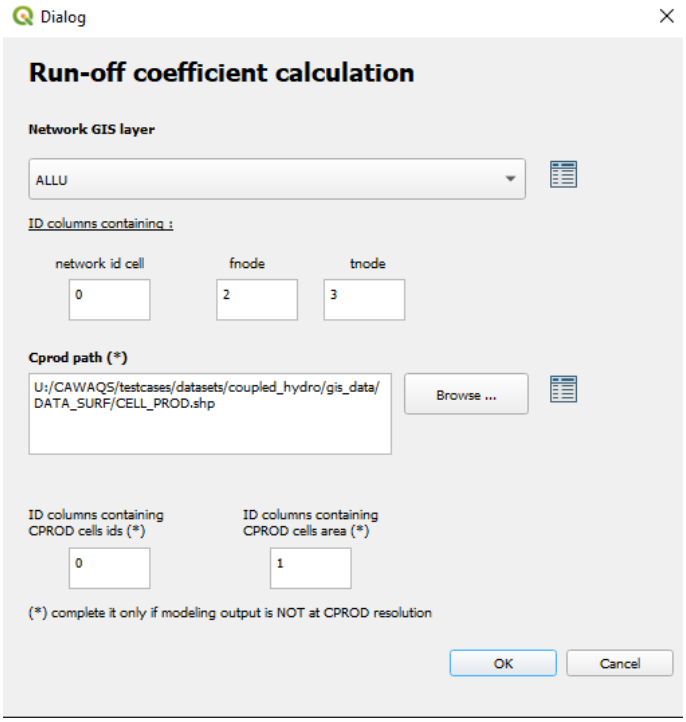


Figure 3.5 – Boîte de dialogue de calcul du coefficient de ruissellement.

Cette boîte de dialogue prend en entrée la résolution du schéma hydrologique de surface à l'échelle du tronçon de rivière et la résolution de surface CaWaQS des mailles de production (bassins versants élémentaires ou `Cprod`). Pour la première, le numéro de colonne contenant les identifiants de tronçons, `Fnode` et `Tnode` doivent être spécifiés. Pour la seconde, les identifiants des champs contenant l'identifiant de la maille de production et l'aire de la maille.

Si la résolution de surface définie dans la configuration CaWaQS-Viz est déjà celle des mailles de production, il ne sera pas nécessaire de renseigner la partie inférieure de la boîte de dialogue.

Deux nouvelles résolutions (*shapefiles*) seront renvoyées par la boîte de dialogue et ajoutées au projet SIG courant :

- "`CATCH`" qui contient les bassins versants drainés par chaque station de mesure ;
- "`CATCH_SURF`", produit de l'intersection entre la résolution de surface définie dans la configuration CaWaQS-Viz et "`CATCH`". Sa table attributaire contient plusieurs champs :
  - "`CATCH_ID`" et "`CATCH_SURF`" : l'identifiant du bassin versant drainé par la station et son aire ;
  - "`ID_OBS`" : l'identifiant de la station de mesure ;
  - "`SURF_ID`" : l'aire de la maille de surface issue de la résolution de surface définie dans la configuration CaWaQS-Viz ;
  - "`SURF_SURF`" : son aire ;
  - "`INTER_ID`" : l'identifiant de la maille d'intersection ;

– "`INTER_SURF`" : l'aire d'intersection.

Ces résolutions sont enregistrées au format temporaire. Il revient à l'utilisateur de les enregistrer pour une réutilisation future. Si CaWaQS-Viz reconnaît ces résolutions dans le projet QGIS courant, elles seront réutilisées pour le calcul des coefficients de ruissellement.

Ces coefficients se trouvent dans la table attributaire de la couche d'observation de débit spécifiée à l'étape de configuration. Deux nouveaux champs sont alors ajoutés : "`QruissSim`" et "`QruissObs`" désignant respectivement le coefficient de ruissellement simulé par CaWaQS et celui observé aux stations de mesure.

3.2 `Compare sim/obs` : comparaison des chroniques observées et simulées

Ce module de calcul permet la comparaison entre simulations et observations à travers plusieurs types de sorties :

- en cliquant sur "`PDF printout`", un fichier `pdf` est généré. Les chroniques simulées et observées sont présentées avec les critères statistiques listés dans la table 3.2 ;
- en cliquant sur "`Interactiv graphics`", un format `html` reproduit interactivement le format de présentation des chroniques du `pdf` dans le navigateur par défaut ;
- en cliquant sur "`Statistical criteria`", la boîte de dialogue "`Statistical Criteria`" (3.1.3) s'ouvre. Les paramètres spécifiés dans la boîte de dialogue "`Compare Sim-Obs`" (dates, compartiment et unité de mesure des observations) sont transférés à la boîte de dialogue "`Statistical Criteria`".

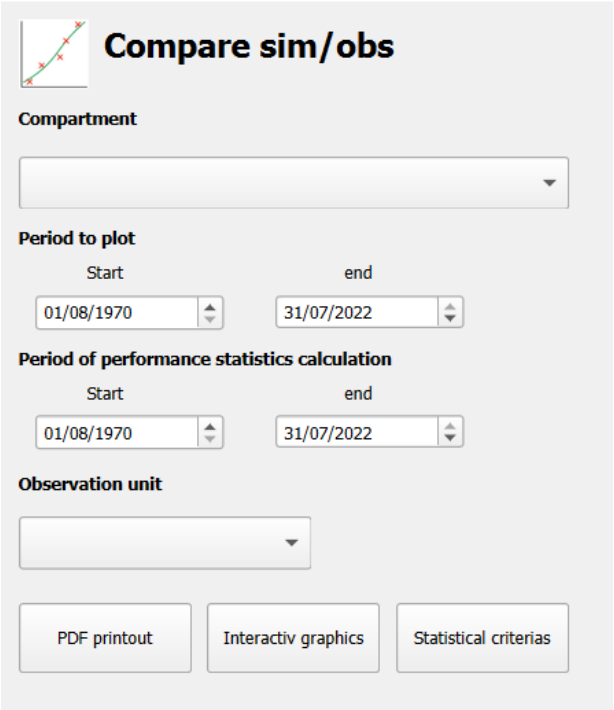


Figure 3.6 – Fenêtre de dialogue "`Compare sim/obs`".

Le type de compartiment (**AQ** = aquifère et **HYD** = réseau hydrographique) est à sélectionner dans la liste déroulante située dans la partie supérieure de la boîte de dialogue (Fig. 3.6). Les périodes d'impression des chroniques et celles de calcul des critères statistiques doivent également être spécifiées. L'unité des observations de débit, utilisée pour le traitement des simulations du compartiment "**HYD**", doit être précisée dans le menu déroulant "**Observation unit**". Elle est optionnelle pour le traitement du compartiment "**AQ**".

Un message d'information sera affiché dans l'interface QGIS à la fin de cette tâche et pointera vers le répertoire de post-traitement. Les fichiers **.pdf** issus de l'analyse du compartiment **HYD** seront nommés "**SIM_OBS_HYD.pdf**" par défaut. Ceux issus de l'analyse du compartiment aquifère seront nommés "**SIM_OBS_AQ.pdf**". Chaque page de ce document correspond à une station de mesure.

3.3 Spatial Map : cartographie spatiale des variables hydrologiques

Cet outil spatialise le bilan hydrique de surface au format **.shp**. Le bilan peut être produit à une fréquence annuelle, mensuelle ou journalière (à préciser dans le champ **frequence**). L'agrégation temporelle peut être réalisée par moyenne, somme, maximum ou minimum (à préciser dans le champ **agregator**).

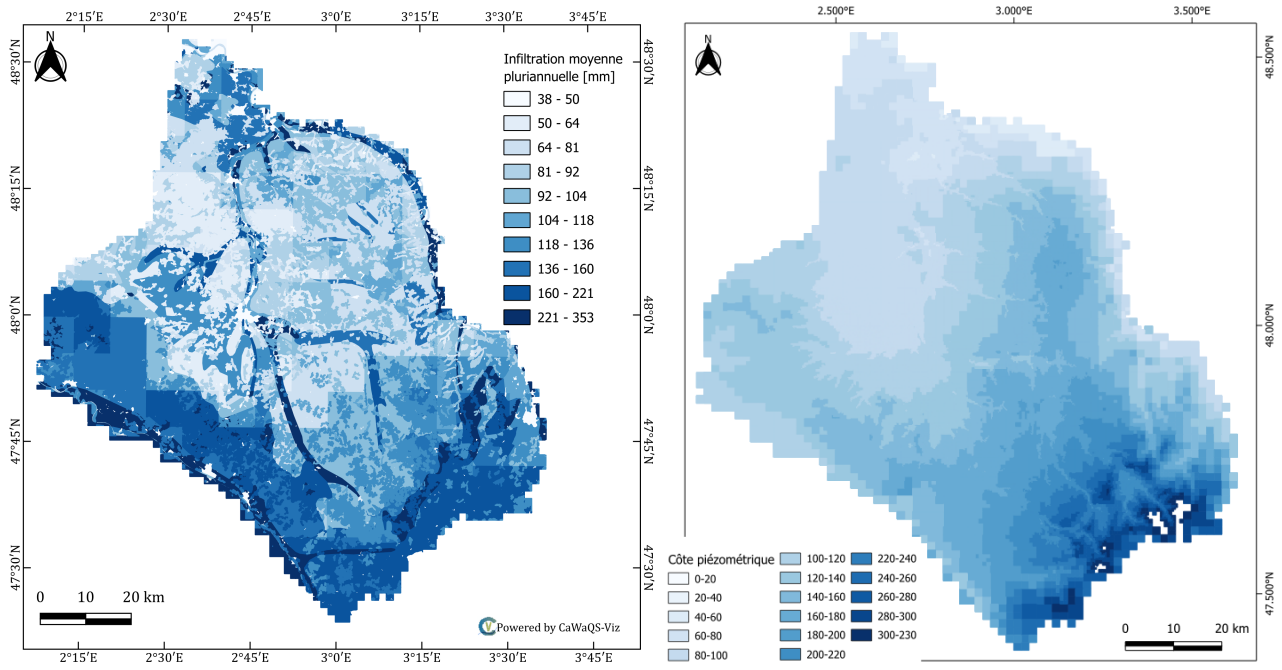


Figure 3.7 – Spatialisation du flux d'infiltration moyen interannuel [mm] (a) et de la piézométrie [$mNGF$] (b) simulés à l'échelle de l'hydrosystème du Loing

Les valeurs prises par la variable hydrologique à spatialiser sont contenues dans la table attributaire de la résolution **.shp** de sortie. La table a pour colonnes les identifiants d'années pour une agrégation annuelle, les identifiants de mois pour une agrégation mensuelle et enfin la date pour une sortie au pas de temps

journalier. Une agrégation pluriannuelle en moyenne est également possible en cochant la case `Pluriannual`, établissant ainsi un bilan pluriannuel ou pluriannuel mensuel.

- `WATBAL` : les variables précipitation, ETR, ruissellement et infiltration (3.7a) ;
- `AQ` : les charges hydrauliques (3.7b), visualisables soit sur une couche de maillage aquifère donnée, soit sur les mailles aquifères affleurantes (maillage unique, créé par CaWaQS-Viz).

Les variables du compartiment `WATBAL` sont exprimées en $mm\,j^{-1}$ ou $mm\,an^{-1}$ selon le mode d'agrégation choisi. Les variables du compartiment `AQ` sont exprimées en mètres.

3.4 `Budget Bar Plot` : visualisation du bilan hydrologique

Les bilans hydrologiques simulés peuvent être visualisés depuis cette boîte de dialogue. Ils sont construits conformément à la période spécifiée. Le champ `Frequences` précise le pas de temps de calcul du bilan hydrique (pluriannuel, annuel, mensuel ou journalier), tandis que `agregator` précise l'agrégateur temporel (moyenne, somme, maximum, minimum). Le bilan hydrique est agrégé spatialement par une moyenne des variables hydrologiques sur l'ensemble des mailles de surface.

À l'exécution de cette boîte de dialogue, une page `HTML` s'ouvre dans votre navigateur et affiche un graphique interactif du bilan hydrologique. Une image statique (cf. Fig. 3.8) de ce même graphique est enregistrée dans le répertoire de post-traitement, dans un sous-dossier `../BUDGET` du répertoire de post-traitement.

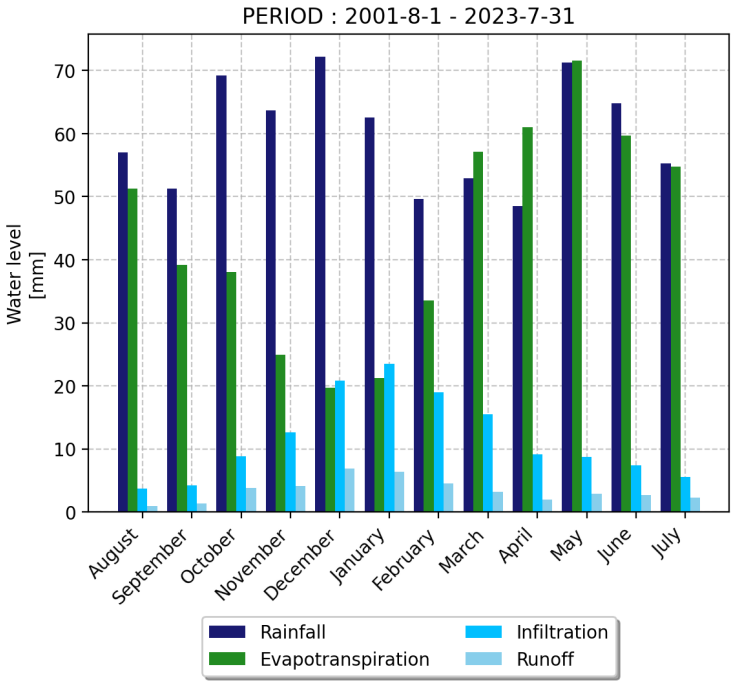


Figure 3.8 – Exemple de sortie graphique du bilan hydrologique pluriannuel mensuel simulé

3.5 `Hydological Regime` : visualisation du régime hydrologique

Les régimes hydrologiques simulés de l'hydrosystème sont construits depuis la boîte de dialogue "**`Hydrological Regime`**" (cf. Figure 3.9). Les régimes hydrologiques sont calculés sur la période spécifiée dans les champs de saisie "**`Start`**" et "**`end`**". La variable à analyser est à préciser dans le menu déroulant "**`Hydrological variable to plot`**". Le type de sortie est choisi par l'utilisateur en cochant la case "**`static plots (PNG)`**" pour des sorties graphiques sous forme d'images individuelles pour chaque point de mesure, "**`static plots (PDF)`**" pour une sortie PDF compilant les régimes hydrologiques de tous les points de mesure, et enfin "**`interractives plots`**" pour une sortie dans le navigateur par défaut sous forme de graphiques interactifs (3.10). Une boîte de dialogue alerte l'utilisateur lorsque le module de calcul a terminé son exécution.

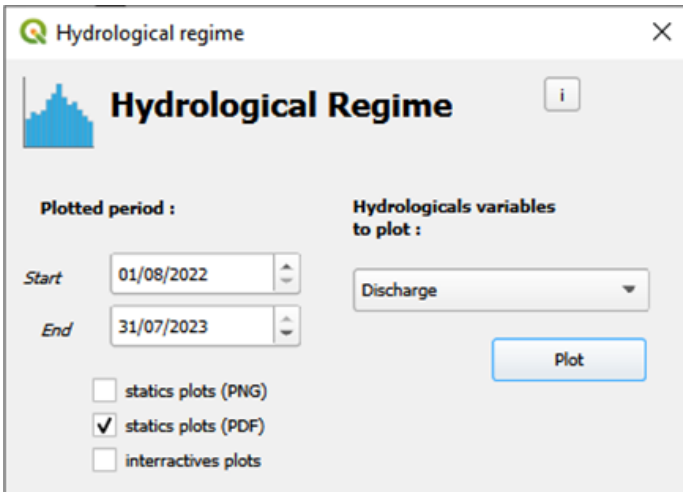


Figure 3.9 – Boîte de dialogue "**`Hydrological Regime`**" pour le calcul des régimes hydrologiques pluriannuels mensuels.

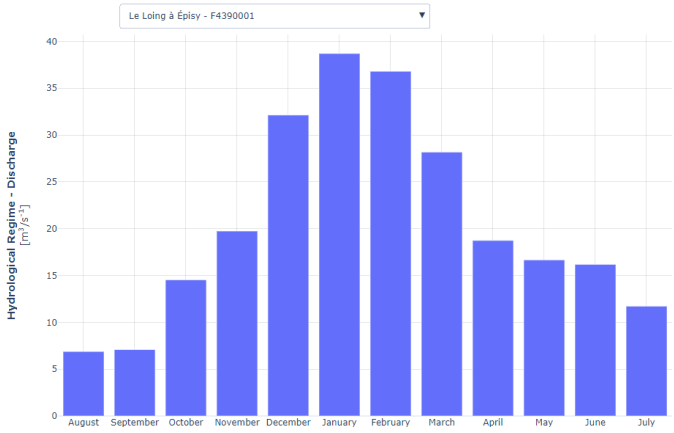


Figure 3.10 – Exemple de sortie graphique du régime hydrologique pluriannuel mensuel simulé

3.6 `Extract Data from a sub area` : extraction des résultats dans un polygone

La boîte de dialogue "**`Extract Data from a Sub Area`**" (Fig. 3.11) permet d'extraire les résultats de simulation sur une zone définie. Cette zone peut être spécifiée soit comme un polygone issu d'un `QgsVectorLayer` chargé dans le projet, soit comme un shapefile importé depuis un répertoire.

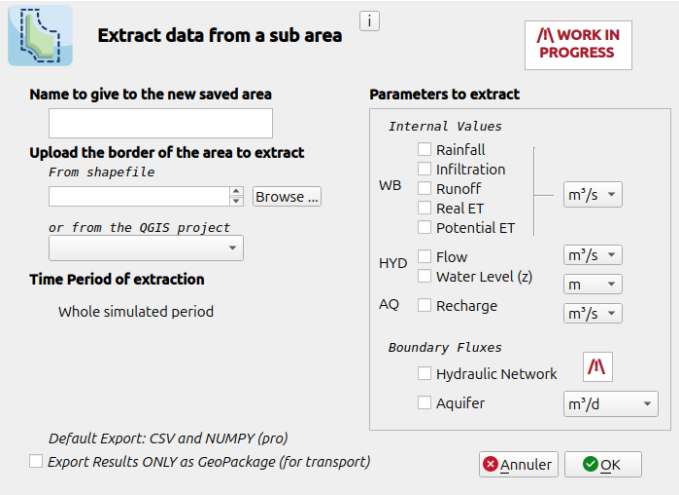


Figure 3.11 – Boîte de dialogue "**`Extract Data from a Sub Area`**".

Deux familles de variables peuvent être extraites, chacune pilotée par son propre jeu de cases à cocher et produisant son propre ensemble de sorties : les *valeurs internes* (Sec. 3.6.4), c.-à-d. les variables simulées des compartiments **`WATBAL`**, **`HYD`** et **`AQ`** à l'intérieur du polygone, et les *valeurs aux limites* (Sec. 3.6.5), c.-à-d. les flux traversant la frontière du polygone. Les deux familles dépendent d'une case à cocher "**`GeoPackage`**" indépendante qui bascule l'ensemble du format de sortie.

3.6.1 Paramètres

Depuis la boîte de dialogue, les paramètres suivants peuvent être spécifiés pour l'extraction :

- le nom du nouveau fichier d'extraction (également utilisé comme nom du dossier créé dans `PostProcessing/DATA`
- le polygone d'extraction : soit un shapefile, soit un `QgsVectorLayer` présent dans le projet ;
- les paramètres à extraire (cases à cocher) ;
- l'unité de mesure, choisie dans la liste déroulante à côté de chaque paramètre (voir Sec. 3.6.3) ;
- si le format de sortie doit être un GeoPackage ou l'organisation par défaut en fichiers séparés (Sec. 3.6.2).

3.6.2 Sorties

Pour chaque extraction, des fichiers de résultats et des `QgsVectorLayer` sont produits. Les fichiers sont enregistrés dans le répertoire de sortie défini dans les paramètres d'analyse principaux, sous `PostProcessing/DATA_EXTRA` qui contient les sous-dossiers suivants :

- **CONFIG/** : fichiers de configuration pour rejouer les simulations WIP ;

- **OUTPUTS/** : fichiers CSV ou GeoPackage contenant les résultats ;
- **TEMP/** : fichiers NumPy utilisés pour rejouer les analyses.

La case à cocher "GeoPackage" n'*ajoute* pas de fichier, elle *remplace* les fichiers séparés. Les deux modes sont mutuellement exclusifs :

- **Décochée (par défaut)** : un CSV par paramètre coché dans OUTPUTS/ et un binaire .npy par paramètre dans TEMP/ (plus un CSV de total sur le polygone par paramètre lorsque la pondération est activée). Aucun GeoPackage n'est écrit.
- **Cochée** : un unique GeoPackage dans OUTPUTS/ et rien d'autre — ni CSV, ni .npy.

3.6.3 Unités

Chaque paramètre porte sa propre unité, choisie dans la liste déroulante à côté de sa case à cocher. Les flux WATBAL, la "Recharge" AQ et le "Flow" HYD sont volumiques (m^3/s ou m^3/d) ; la "Water Height" HYD est une longueur (m ou cm) et n'est donc pas soumise à la conversion volumique par jour. Les flux souterrains (aux limites) AQ disposent de leur propre liste déroulante d'unités, décrite en Sec. 3.6.5.

3.6.4 Paramètres — valeurs internes

Les variables suivantes peuvent être extraites pour la surface (compartiment WATBAL de CaWaQS) :

- "Rainfall" ;
- "Real EvapoTranspiration (ET)" ;
- "Infiltration" ;
- "Runoff" ;
- "Potential EvapoTranspiration (ET)".

Pour le réseau hydrographique (HYD) :

- "Flow" ;
- "Water Height" ;

Pour l'aquifère (AQ) :

- "Recharge".

3.6.4.1 Le GeoPackage des valeurs internes

Lorsque la case "GeoPackage" est cochée, tous les compartiments cochés sont regroupés dans un unique fichier, <area>_InternalValues_<years>.gpkg, écrit dans OUTPUTS/. Il contient :

- une couche vectorielle cells_<compartment> par maillage (par ex. cells_WATBAL, cells_AQ, cells_HYD) portant la géométrie masquée — les tronçons HYD sont découpés à la frontière du polygone ;
- une table daily_values comportant une ligne par tuple (compartiment, maille, date, paramètre) ; elle est jointe à une couche de maillage sur la paire (compartment, cell_id) ;
- une table provenance comportant une ligne par compartiment ;
- une table polygon_total_<compartment>_<param> par paramètre, lorsque la pondération est activée.

3.6.5 Paramètres — valeurs aux limites

- flux souterrains (horizontaux) de l'"`Aquifer`" : les flux traversant les faces verticales des mailles aquifères situées sur la frontière du polygone ;
- "`Hydraulic network`" WIP : débit des tronçons traversant les limites du masque.

3.6.5.1 Convention d'unité et de signe

La case à cocher "`Subsurface flows`" possède sa propre liste déroulante d'unités, offrant deux options qui diffèrent par leur *nature*, et pas seulement par leur échelle :

- `m3/d` (par défaut) : un **débit** journalier (le `m3/s` brut multiplié par 86 400), donnant une valeur par jour ;
- `m3/month` : le **volume** total réel (`m3`) ayant traversé chaque limite au cours de chaque mois calendaire. L'axe temporel est ré-échantillonné à une ligne par mois, chaque valeur étant la somme, sur les jours de ce mois, des `m3/s` $\times$ 86 400 journaliers. Les longueurs réelles des mois (28, 29, 30 ou 31 jours) sont respectées.

> En mode `m3/month`, un premier ou dernier mois partiel ne totalise **que ses jours simulés**. Un premier ou dernier mois dont le volume paraît faible comparé à ses voisins est donc correct, et non une erreur.

La convention de signe accompagne les données — dans la ligne d'en-tête du CSV et dans la table `provenance` du GeoPackage : **une valeur positive signifie un flux entrant dans la maille**.

3.6.5.2 Mode par défaut : le CSV des flux de faces

Lorsque la case "`GeoPackage`" est décochée, les flux aux limites sont écrits dans un CSV de flux de faces séparé. Les faces sont nommées selon la convention `nCell(ID_ABS)_direction_unit`, par exemple `3323_west_m3d`. C'est le **seul** mode qui expose le détail *brut* par face, c.-à-d. la valeur propre de chaque face prise séparément.

3.6.5.3 Mode GeoPackage : le GeoPackage des limites AQ

Le GeoPackage des limites AQ est exclusif de la même manière que celui des valeurs internes : lorsque la case est cochée, un unique `<area>_AqBoundary_<years>.gpkg` est écrit *à la place* du CSV de flux de faces séparé. Il contient :

- une couche de géométrie `cells_AQ_layer<id>` par couche aquifère ;
- une table de valeurs — `daily_values` en mode `m3/d`, `monthly_values` en mode volume total `m3/month`.

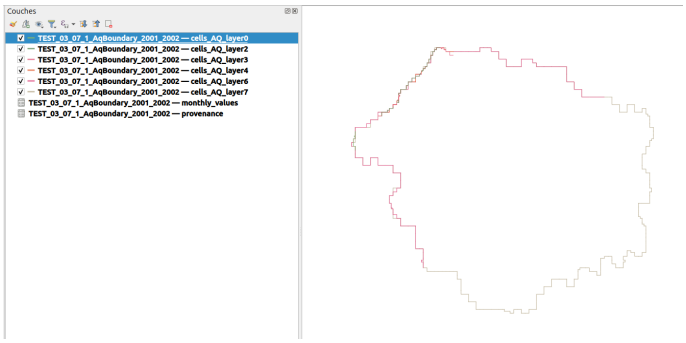


Figure 3.12 – "`AQ boundary geopackage`".

Les couches de géométrie comme la table de valeurs portent une colonne `faces` : la liste, séparée par des virgules, des directions cardinales que chaque maille de bordure touche, par exemple `'north,west'`. Chaque ligne reste **une série nette par maille** ; la colonne `faces` indique simplement quelles directions y ont contribué. La colonne `faces` apparaît aussi dans la table attributaire de la couche de bordures QGIS enregistrée dans le projet, indépendamment du mode GeoPackage.

Colonnes de structure par face. À l'intérieur du fichier GeoPackage uniquement, sept colonnes supplémentaires répartissent ce résumé sur des champs interrogeables. Elles apparaissent à la fois sur les couches de géométrie `cells_AQ_layer<id>` et sur la table de valeurs, mais **pas** sur la couche de bordures QGIS enregistrée, qui ne conserve que `faces`.

- `n_faces` : nombre de côtés cardinaux que borde la maille (1 à 3) ;
- `face1_orient/face1_outid` … `face3_orient/face3_outid` : deux colonnes par emplacement de face.

`faceN_orient` est la N-ième direction bordée ; les emplacements au-delà de `n_faces` sont laissés vides. `faceN_outid` n'est renseigné **que** lorsque le flux de cette face provient de la maille extérieure — le cas *coarse-inside*, où la valeur reportée provient des voisines extérieures plus petites, dont les identifiants sont alors joints par des virgules dans le champ. Il reste vide pour une lecture normale de la face propre.

3.6.6 Utilisation de l'extraction depuis un notebook

Les deux familles de la boîte de dialogue sont également disponibles depuis un notebook (chapitre 4) via l'API client :

- `client.mask_internal_values(...)` pour les valeurs internes ;
- `client.mask_aq_boundary(...)` pour les flux aux limites de l'aquifère.

Les cases à cocher de la boîte de dialogue sont alors remplacées par des arguments explicites.

3.6.6.1 Valeurs internes : les tuples `specs`

`mask_internal_values` prend une liste `specs` dont chaque élément est un 4-uplet `(compartment, outtype, param, unit)` :

- `compartment` : `"WATBAL"`, `"HYD"` ou `"AQ"`. Le compartiment doit être chargé par le client (`IDS_COMPARTMENTS` : 1 = **`AQ`**, 2 = **`HYD`**, 3 = **`WATBAL`**) ;
- `outtype` : le binaire CaWaQS dans lequel le paramètre est lu (`MB`, `Q` ou `H`) ;
- `param` : un nom d'enregistrement de ce binaire ;
- `unit` : l'unité de sortie de *ce* tuple. Chaque tuple porte sa propre unité ; il n'y a pas d'argument d'unité global.

Le Tab. 3.3 liste toutes les combinaisons acceptées.

compartment	outtype	param	unit
WATBAL	MB	rain, etp, runoff, inf, etr, direct_sout, stocksoil, stockinf, stockruiss, error	m3/s, m3/j
HYD	Q	discharge	m3/s, m3/j
HYD	H	water_height, water_level	m, cm
AQ	H	piezhead	m, cm
AQ	MB	h_end, dv_dt, flux_x_one, flux_x_two, flux_y_one, flux_y_two, flux_z_one, flux_z_two, recharge, uptake, flux_direchlet, flux_neumann, surf_overflow, flux_riv_to_aq, err, err_rel	m3/s, m3/j

Tableau 3.3 – Tuples (`compartment, outtype, param, unit`) acceptés par `mask_internal_values`.

Les cases à cocher de la Sec. 3.6.4 correspondent aux tuples suivants : "**`Rainfall`**" = `rain`, "**`Real EvapoTranspiration (ET)`**" = `etr`, "**`Potential EvapoTranspiration (ET)`**" = `etp`, "**`Infiltration`**" = `inf`, "**`Runoff`**" = `runoff`, "**`Flow`**" = `discharge`, "**`Water Height`**" = `water_height`, "**`Recharge`**" = `("AQ", "MB", "recharge", ...)`. Les autres paramètres du tableau ne sont disponibles que depuis un notebook.

```
MASK_SPECS = [
    ("WATBAL", "MB", "runoff",        "m3/j"),
    ("HYD",    "Q",  "discharge",     "m3/s"),
    ("HYD",    "H",  "water_level",   "m"),
    ("AQ",     "MB", "recharge",      "m3/j"),
    ("AQ",     "MB", "surf_overflow", "m3/j"),
]
```

Code 3.1 – Exemple de specs pour `mask_internal_values`.



L'unité doit appartenir à la bonne famille. `water_height`, `water_level` et `piezhead` n'acceptent que `m` ou `cm` ; tous les autres paramètres n'acceptent que `m3/s` ou `m3/j`. Un mauvais appariement lève une `TypeError`. Les jetons `m3/mois`, `m3`, `mm/j` et `l/s` ne sont **pas** acceptés pour les valeurs internes.

Quelques comportements à connaître :

- les tuples **`AQ`** sont résolus sur la couche aquifère *affleurante* (supérieure), et non couche par couche. C'est le bon support pour `recharge` et `surf_overflow`. Pour les flux latéraux à travers la frontière du polygone, utilisez **`mask_aq_boundary`** (ci-dessous) plutôt que `flux_x_*/flux_y_*`, qui renverraient toutes les mailles *à l'intérieur* du polygone ;

- `weighted=True` est ignoré pour les unités de longueur (`m`, `cm`) et pour tous les tuples **HYD** : les tronçons contribuent toujours par leur valeur brute ;
- `h_end` et `err_rel` sont acceptés avec une unité volumique, mais ne sont pas des flux physiques : à interpréter avec prudence.

3.6.6.2 Valeurs aux limites : `mask_aq_boundary`

mask_aq_boundary ne prend **pas** de `specs` : il lit toujours les quatre flux de faces latérales du binaire `AQ_MB` (`flux_x_one` = ouest, `flux_x_two` = est, `flux_y_one` = sud, `flux_y_two` = nord) sur les mailles situées sur la frontière du polygone, séparées par couche aquifère. Le seul argument d'unité est `unit` :

- `"m3/j"` (par défaut) : débit journalier en m^3/d — l'option m^3/d de la boîte de dialogue ;
- `"m3"` : volume total par mois calendaire, écrit dans une table `monthly_values` — l'option $m^3/month$ de la boîte de dialogue ;
- `"m3/mois"` (notebook uniquement) : un débit *mensuel moyen*, c.-à-d. chaque valeur journalière multipliée par $86\,400 \times 365{,}25/12$, en conservant une ligne par jour.

```
result = client.mask_aq_boundary(
    polygon=polygon, polygon_crs=polygon_crs,
    syear=START_YEAR, eyear=END_YEAR,
    output_dir=CWV_output_directory, temp_dir=TEMP_DIR,
    area_name="MY_AREA_AQ_BOUNDARY",
    write_geopackage=True,
    unit="m3",          # "m3/j" (defaut), "m3" ou "m3/mois"
)
```

Code 3.2 – Exemple d'appel de `mask_aq_boundary`.



Les deux opérations nécessitent les maillages **AQ** : chaque couche aquifère référencée par la configuration géométrique doit figurer dans `layer_paths`, et `1` doit être inclus dans `IDS_COMPARTMENTS`. Sinon, la cellule du notebook affiche `Skipped`.

CHAPITRE 4

Utiliser CaWaQS-Viz depuis un notebook (sans QGIS)

Sommaire

4.1 Utiliser CaWaQS-Viz depuis un notebook (sans QGIS) - pré-requis 35
4.2 Mettre en place l'environnement 35
4.3 Exécuter sur un serveur distant 35
4.4 Choisir un modèle 36
4.5 Flux de travail 36

4.1 Utiliser CaWaQS-Viz depuis un notebook (sans QGIS) - pré-requis

Le backend scientifique de CaWaQS-Viz, `HydrologicalTwinAlphaSeries` (HTAS), peut être piloté directement depuis un notebook Python, **sans QGIS**. Cela met en œuvre la même API publique que celle utilisée par les boîtes de dialogue QGIS, sur une véritable sortie de simulation CaWaQS. Au-delà de la commodité, ces notebooks garantissent que le backend reste indépendant de QGIS : tout ce qui s'exécute aujourd'hui depuis un simple notebook Python s'exécutera aussi sur le futur serveur CaWaQS-Viz (chapitre 7). Les notebooks se trouvent sous `interface/notebooks/`.

- une copie locale de `cawaqsviz` avec le sous-module `HydrologicalTwinAlphaSeries` renseigné (Sec. 1.1) ;
- un répertoire de sorties de simulation CaWaQS lisible : les binaires sous le répertoire de sorties CaWaQS, le `.json` `ConfigGeometry` livré avec la simulation, et les `shapefiles`/`geopackages` correspondants.

4.2 Mettre en place l'environnement

La méthode recommandée est un environnement `pixi` dédié qui installe `jupyterlab` et une installation éditable de HTAS, mais *pas* QGIS. Depuis la racine du dépôt :

```
pixi install -e notebook          # cree l'environnement notebook (une fois)
pixi run -e notebook notebook     # lance Jupyter Lab, enracine dans interface/notebooks/
```

Jupyter Lab s'ouvre dans votre navigateur ; double-cliquez sur un notebook et exécutez-le.

`pixi` n'est qu'une commodité. HTAS est un paquet Python pur standard (pas de QGIS, pas de dépendances propres à conda), si bien que n'importe quel environnement virtuel disposant de `jupyterlab` convient :

```
# venv + pip
python -m venv .venv && source .venv/bin/activate
pip install jupyterlab
pip install -e external/HydrologicalTwinAlphaSeries
jupyter lab interface/notebooks/
```

L'installation éditable de `external/HydrologicalTwinAlphaSeries` tire les dépendances d'exécution de HTAS (`geopandas`, `numpy`, `pandas`, `matplotlib`, `plotly`, `scipy`, `shapely`). La seule chose que `pixi` ajoute est un environnement entièrement figé et reproductible.

4.3 Exécuter sur un serveur distant

Lorsque la simulation réside sur un serveur de calcul, on souhaite que le notebook s'*exécute* là-bas (à côté des données) mais qu'il s'*édite* sur son ordinateur portable. La méthode recommandée est l'extension **Remote-SSH** de VS Code, qui démarre le noyau sur le serveur et ouvre le `.ipynb` comme un onglet d'éditeur normal — pas de navigateur, pas de jeton, pas de tunnel. Enregistrez l'environnement une fois, sur le serveur, depuis la racine du dépôt :

```
pixi run -e notebook register-kernel
```

Cela fait apparaître un noyau nommé « CaWaQS (pixi notebook) » dans le sélecteur de noyaux de VS Code (sélectionnez Kernel>Jupyter Kernel> CaWaQS (pixi notebook)). Vous vous connectez ensuite avec "`Remote-SSH: Connect to Host`", ouvrez le `.ipynb`, sélectionnez ce noyau et exécutez les cellules.

Si VS Code n'est pas disponible, un tunnel SSH constitue une solution de repli fonctionnelle : démarrez `jupyter lab -no-browser` sur le serveur, redirigez le port avec `ssh -L 8888:localhost:8888`, et ouvrez l'URL affichée localement. Voir `interface/notebooks/README.md` pour la marche à suivre complète.

4.4 Choisir un modèle

Le dossier `interface/notebooks/` livre **deux modèles**. Tous deux partagent la même cellule de paramètres et le même contrat `layer_paths` — ils ne diffèrent que par la surface HTAS qu'ils appellent :

- `template_client.ipynb` — l'API grossière, calquée sur les boîtes de dialogue (`HydrologicalTwinClient`). Copiez-le pour **reproduire ce que fait une boîte de dialogue QGIS** (par ex. `budget_barplot`, `spatial_map_aq`, `mask_hyd_boundary`). C'est également la surface qu'exposera un futur serveur HTTP ;
- `template_developer.ipynb` — l'API de bas niveau, à base de verbes (`HydrologicalTwin`, verbes `fetch` / `mask` / `transform` / `render`). Copiez-le pour **construire votre propre pipeline**, inspecter des tableaux intermédiaires, ou appeler une primitive pas encore disponible dans le client.

Règle empirique : « *un tracé qui correspond à une boîte de dialogue QGIS* » → **client** ; « *ma propre chaîne d'opérations* » → **developer**. Les deux surfaces sont les deux *portes d'entrée* de HTAS, décrites dans le chapitre développeur (Sec. 7.4).

4.5 Flux de travail

1. **Copiez le modèle** — ne modifiez jamais un modèle en place. Cliquez droit dessus → "`Duplicate`" et renommez la copie (par ex. `seine_2023.ipynb`).
2. **Ne modifiez que la cellule de paramètres.** Le reste du notebook est conçu pour s'exécuter sans modification.
3. **Effacez les sorties avant de committer.** Les sorties alourdissent les diffs et peuvent divulguer des chemins de votre machine. Depuis la racine du dépôt :

```
jupyter nbconvert --clear-output --inplace interface/notebooks/<your_notebook>.ipynb
```

CHAPITRE 5

Configuration manuelle du projet (avancé)

Sommaire

5.1 Contenu du projet QGIS utilisé pour le traitement des résultats de simulation CaWaQS 38
5.2 Configurer la géométrie du projet depuis un projet QGIS existant 40
5.3 Configurer les résolutions depuis un fichier de configuration `.json` 44



La configuration du projet est désormais **automatisée** : lors d'un premier lancement normal, le plug-in détecte les sorties CaWaQS et renseigne la configuration des géométries à votre place. La voie manuelle décrite dans ce chapitre est une **solution de repli** pour les utilisateurs avancés — lorsque la détection automatique ne parvient pas à résoudre le projet, ou lorsqu'une configuration doit être construite ou ajustée à la main. Les nouveaux utilisateurs n'en ont pas besoin pour démarrer.

La partie inférieure de la boîte de dialogue "`Settings`" permet la configuration manuelle du plug-in. Les éléments suivants doivent être définis :

1. un nom de projet ;
2. les compartiments modélisés ;
3. une configuration des maillages de couches SIG ;
4. les répertoires de sorties de modélisation et d'observations [1] si des points d'observation sont définis dans le projet.

Le nom du projet servira à nommer un sous-dossier créé dans le répertoire de post-traitement. Les sorties graphiques de l'application CaWaQS-Viz seront placées dans ce dossier de post-traitement.

La configuration des géométries peut aussi être importée depuis un fichier de configuration `.json` (en cliquant sur "`Import a geometry configuration from a .json`") ou définie manuellement *via* l'option "`Select resolutions from the current project`".



Notez que toute configuration manuelle du projet et des géométries sera exportée au format `.json` vers le répertoire de post-traitement à l'exécution de la boîte de dialogue "`Settings`".

5.1 Contenu du projet QGIS utilisé pour le traitement des résultats de simulation CaWaQS

Les `shapefiles` des maillages du modèle CaWaQS

Le projet QGIS utilisé pour le traitement des simulations avec l'utilitaire CaWaQS-Viz doit contenir :

- les maillages (résolutions) des compartiments au format "`shapefile`" du modèle (référez-vous à la documentation CaWaQS [2] pour identifier les résolutions de sortie du compartiment "`WATBAL`"). Les types de résolutions CaWaQS à importer dans le projet SIG pour le traitement des sorties de modélisation dépendent des compartiments réellement modélisés ;
- les points d'observation des compartiments à définir dans l'extension CaWaQS-Viz.

1. Les répertoires d'observations de débit et de piézométrie sont supposés se trouver dans le même répertoire parent.
2. Terminologie des géométries CaWaQS. Pour plus d'informations sur les différents types de maillages, référez-vous à la documentation du modèle (*i.e.* guide utilisateur et guide technique), disponible sur le dépôt GitLab https://gitlab.com/cawaqs/cawaqs.

	Compartiment de surface (WATBAL)	Compartiment hydraulique (HYD)	Compartiment non saturé (NSAT)	Compartiment aquifère (AQ)
Résolution CaWaQS	Maille de surface unitaire (`ele_bu`)[3] ou maille de production (`Cprod`)	Élément de rivière (`ele_musk`)	Grille de zone non saturée	Maillages de toutes les couches aquifères
Champs requis dans la table attributaire	Identifiants de mailles	Identifiants SIG des éléments de rivière	Identifiants de mailles	Identifiants absolus de mailles

Tableau 5.1 – Résolutions CaWaQS pouvant être importées comme résolution de compartiment.



Les noms des couches SIG du projet correspondant aux aquifères doivent être identiques à ceux définis dans la configuration CaWaQS afin de garantir le bon comportement de l'application.

Les `shapefiles` des points d'extraction

L'extraction des débits et charges hydrauliques simulés peut être réalisée aux points d'extraction. Deux types de points d'extraction peuvent être définis :

- les points d'extraction simples, qui n'incluent pas de chronique de mesure ;
- les points d'observation, qui sont des points d'extraction disposant de données de mesure.

Les points d'extraction et d'observation sont contenus dans deux fichiers **`shapefile`** distincts dont les tables attributaires contiennent les champs listés dans la table 5.2.

TYPE DE POINT	CONTENU DE LA TABLE ATTRIBUTAIRE	SORTIES
Point d'extraction	• Nom du point • Identifiant de la maille rattachée [4], identifiant de la couche aquifère (en cas d'extraction dans l'aquifère)	• Hydrogrammes et courbes piézométriques simulés au point d'extraction
Point d'observation	• Nom du point • Identifiant de la maille rattachée [4]	• Hydrogrammes et courbes piézométriques simulés et observés • Critères objectifs comparant observations et simulations

Tableau 5.2 – Structure des couches SIG des points d'extraction et sorties graphiques associées



Les `shapefiles` d'observation doivent être importés dans le projet et spécifiés dans la configuration CaWaQS-Viz si les compartiments HYD et/ou AQ sont traités. Sans observations, l'extension ne sera pas fonctionnelle.

5.2 Configurer la géométrie du projet depuis un projet QGIS existant

La configuration géométrique de l'application CaWaQS-Viz peut être définie depuis un projet QGIS existant via la boîte de dialogue "`Application configuration from an existing project`" dans la boîte de dialogue des paramètres de l'application.

La fenêtre est organisée en deux sections :

- à gauche, les compartiments CaWaQS sont listés (WATBAL, HYD, AQ, NSAT),
- à droite, les paramètres à définir sont listés pour produire les sorties graphiques (débit et piézométrie) aux points d'observation (points d'extraction avec observations) ou aux points d'extraction (sans observations).

Si les champs de configuration de l'application sont laissés vides, le compartiment concerné n'est pas inclus dans la configuration géométrique.

Les champs des compartiments sont configurés comme suit :

Le compartiment de surface (`""WATBAL COMPARTMENT""`) :

- "`Surface resolution`" : résolution de sortie du bilan hydrique de surface (CPROD ou elebu) ;
- "`cell id`" : le numéro de colonne dans la table attributaire correspondant à l'identifiant de la maille ;
- "`cell area`" : le numéro de colonne dans la table attributaire correspondant à l'aire de la maille (paramètre optionnel).

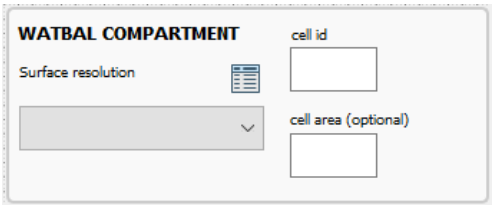


Figure 5.1 – Champs de configuration du compartiment WATBAL

Le compartiment hydraulique ("`HYD COMPARTMENT`")

- "`Hyd resolution`" : nom de la résolution de sortie du compartiment "`HYD`" (éléments de rivière) ;
- "`GIS Id`" : le numéro de colonne dans la table attributaire correspondant à l'identifiant SIG des mailles



Figure 5.2 – Champs de configuration du compartiment HYD

Le compartiment non saturé ("`NSAT COMPARTMENT`") :

- "`Unsatured zone resolution`" : nom de la résolution de sortie du compartiment "`NSAT`" ;
- "`cell id`" : le numéro de colonne dans la table attributaire correspondant à l'identifiant de la maille.

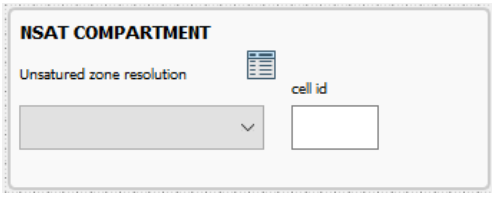


Figure 5.3 – Champs de configuration du compartiment NSAT

Le compartiment saturé ("`AQ COMPARTMENT`")

- "`Aquiferes resolutions`" : les couches SIG constituant le maillage souterrain. La couche sélectionnée est ajoutée en cliquant sur "`Add`". Les couches doivent être ordonnées de la plus haute à la plus basse à l'aide des boutons "`up`" et "`down`". Enfin, une couche peut être retirée de la liste en cliquant sur "`remove`";
- "`cell id abs`" : identifiant de la colonne de la table attributaire contenant l'identifiant absolu de la maille. Cet identifiant peut être homogène entre les couches SIG : dans ce cas, "`homogeneous ID ABS`" doit être sélectionné. Sinon, cliquez sur "`heterogeneous ID ABS`" pour ouvrir des boîtes de dialogue permettant de spécifier, pour chaque couche, la colonne contenant l'ID ABS des mailles (cf. Fig 5.4).

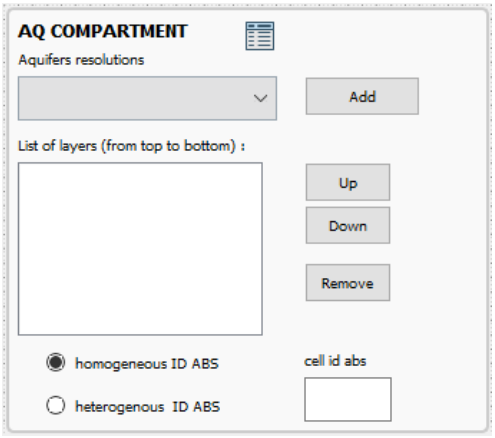


Figure 5.4 – Champs de configuration du compartiment AQ

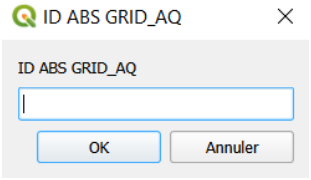


Figure 5.5 – Fenêtre de sélection de l'identifiant du champ contenant l'"`ID ABS`" dans la table attributaire dans le cas d'identifiants de champs hétérogènes entre les couches

Quelques points de vigilance :

- **L'identifiant de la première colonne d'une table attributaire commence à 0**. L'icône permet de visualiser les identifiants des champs listés dans la table attributaire de la résolution sélectionnée dans la liste déroulante ;
- le code (par ex. code BSS) d'un point d'observation doit être **unique** parmi tous les points d'observation ;
- La configuration géométrique des compartiments cochés dans la boîte de dialogue de paramétrage du projet doit être définie dans la boîte de dialogue de configuration des géométries.

Points d'extraction - "`Hydrological and Hydrological plots`"

Les points d'extraction peuvent être de deux types :

- les points d'extraction simples, qui permettent de visualiser les variables simulées en ces points ;
- les points d'observation, qui permettent de comparer les variables simulées aux données d'observation en ces points.

La configuration des points d'extraction nécessite de spécifier dans les champs "`GIS Layer`" le nom de la couche du projet QGIS contenant les points d'observation et les points d'extraction selon le compartiment `CaWaQS` concerné. Les identifiants de champs à spécifier sont les suivants :

Points d'extraction du compartiment "HYD"

Points d'extraction

- "name" : nom du point d'observation ;
- "ID GIS" : identifiant SIG du point d'observation auquel le point d'observation doit être rattaché (champ optionnel)

Points d'observation

- "code" : code d'identification du point d'observation, doit être unique pour chaque point. Le fichier d'observation du point doit porter le même nom avec l'extension .dat ;
- les champs "name" et "ID GIS" sont également spécifiés pour la configuration des points d'observation

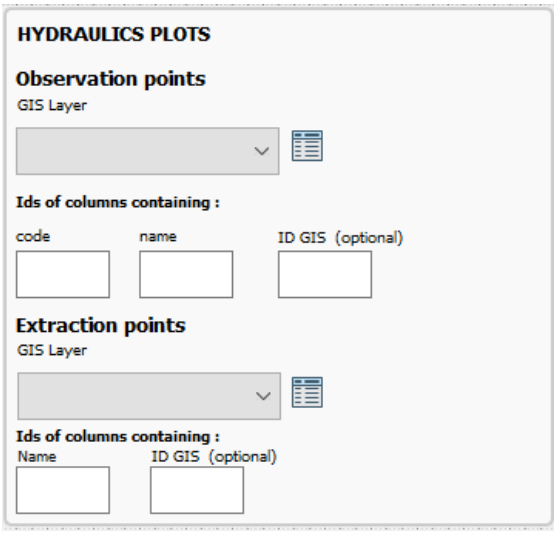


Figure 5.6 – Champs de configuration des points d'extraction du compartiment "HYD"

Points d'extraction du compartiment "AQ"

Points d'extraction

- "name" : nom du point d'observation ;
- "ID LAYER" : identifiant de la couche aquifère
- "ID GIS" : identifiant SIG du point d'observation auquel le point d'observation doit être rattaché (champ optionnel)

Points d'observation

- "code" : code d'identification du point d'observation, doit être unique pour chaque point. Le fichier d'observation du point doit porter le même nom avec l'extension .dat ;
- les champs "name" et "ID GIS" sont également spécifiés pour la configuration des points d'observation

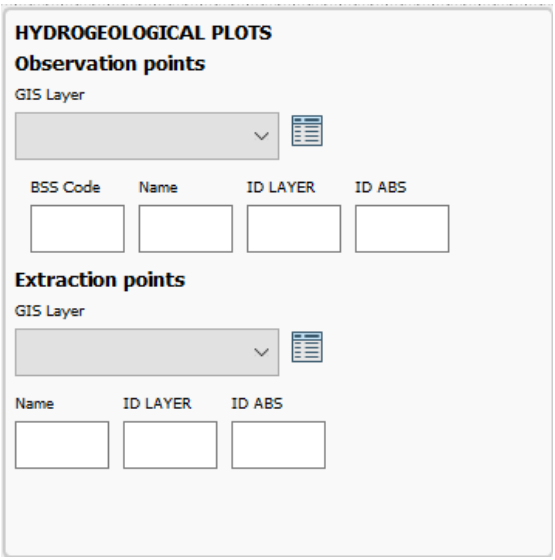


Figure 5.7 – Champs de configuration des points d'extraction du compartiment "AQ"

Les points d'extraction doivent être contenus dans un fichier .shp différent de celui des points d'observation. Les points ne peuvent pas être contenus dans le même fichier s'ils sont extraits de deux compartiments différents.



Un enregistrement préalable du fichier de configuration des géométries peut être effectué en spécifiant un chemin de fichier dans le champ situé en bas à gauche de la boîte de dialogue. Ce fichier (cf. Code 5.1) sera enregistré à l'exécution de la boîte de dialogue "**Settings**" si le projet configuré n'existe pas encore dans le répertoire de post-traitement spécifié. Plusieurs configurations de projet peuvent être enregistrées dans le même répertoire de post-traitement, à condition qu'elles ne portent pas le même nom de projet.

5.3 Configurer les résolutions depuis un fichier de configuration `.json`

La configuration géométrique des résolutions peut être spécifiée via un fichier `.json` selon la structure suivante :

```
{   // liste des compartiments CaWaQS a analyser (1 = AQ, 2 = HYD, 3 = WATBAL)
    "ids_compartment": [],

    "resolutionNames":
    {
        "1":
            [[""]], // liste des noms des maillages aquiferes dans le projet QGIS
        "2" :
            [[""]], // nom du maillage riviere dans le projet QGIS
        "3" :
            [[""]] // nom du maillage de surface dans le projet QGIS
        "4" :
            [[""]] // nom du maillage de zone non saturee dans le projet QGIS
        },
    "ids_col_cell": { // "id compartiment" : "numero de colonne de l'id de maille"
        "1" : "",  // "AQ id" : "numero de colonne de l'id absolu"
        "2" : "",  //"HYD id" : "numero de colonne de l'id SIG"
        "3" : ""   // "WATBAL id" : "numero de colonne de l'id de maille"
        },
    "obsNames": {
    // "id compartiment" : "nom du shp d'obs associe dans le projet QGIS"
        "1": "",
        "2" : ""
        },
    "obsIdsColCells": {
    // "id compartiment" : "numero de champ du code du point de mesure"
        "1": "",
        "2": ""
        },
    "obsIdsColNames": {
    // "id compartiment" : "numero de champ du nom du point de mesure"
        "1": "",
        "2" : ""
        },
```

```
    "obsIdsColLayers": {
    // "id compartiment" : "numero de champ de l'identifiant de couche aquifere"
        "1": "",
        "2" : null
        },
    "obsIdsCell": {
    // "id comp." : "numero de champ de l'id absolu ou sig de la maille rattachee a l'obs"
        "1" : "",
        "2" : null
    },
     "extNames": {
    // "id compartiment" : "nom du shp d'extraction associe dans le projet QGIS"
        "2": "",
        "1": ""
    },
    "extIdsColNames": {
    // "id compartiment" : "numero de champ contenant le nom du point d'extraction"
        "2": "",
        "1": ""
    },
    "extIdsColLayers": {
    // "id compartiment" : "numero de champ contenant l'identifiant de couche aquifere"
        "2": null,
        "1": ""
    },
    "extIdsColCells": {
    // "id compartiment" : "numero de champ contenant l'identifiant de maille"
        "1": "",
        "2": "",
    }
}
}
```

Code 5.1 – Contenu d'un fichier de configuration des géométries

Deuxième partie

Guide du développeur

CHAPITRE 6

Concepts clés pour les développeurs

Sommaire

6.1 Rattacher les points d'observation aux mailles 48
6.2 Le compartiment HYD : `ID_ABS` contre `ID_GIS` 48

Ce chapitre rassemble un petit ensemble de **règles comportementales** que le code suit silencieusement — des décisions qui ne sont pas évidentes à la lecture d'une seule fonction, mais qu'un nouveau développeur doit connaître avant de toucher aux données circulant entre CaWaQS et CaWaQS-Viz. C'est le lieu naturel du savoir « pourquoi le code fait-il *cela* ? » ; la visite guidée de l'architecture et les recettes concrètes d'édition suivent au chapitre 7.

6.1 Rattacher les points d'observation aux mailles

Un point d'observation (une station de jaugeage, un piézomètre, …) doit être rattaché à exactement une maille avant que ses valeurs simulées puissent être lues. CaWaQS-Viz décide quelle maille utiliser de **deux façons différentes**, et le choix se fait *par observation*, pas globalement :

- **Par identifiant (préféré).** Si la couche d'observation porte un identifiant de maille explicite — une colonne `ID_ABS` configurée dans la table attributaire de l'observation (`.dbf`) — cet identifiant est utilisé directement. C'est la voie faisant autorité : le rattachement est exactement ce que déclarent les données, indépendamment de la géométrie.
- **Par géométrie (repli).** Si aucun identifiant de ce type n'est fourni, le code se rabat sur une recherche de la **maille la plus proche** : il prend la géométrie ponctuelle de l'observation et trouve la maille la plus proche de la couche de maillage concernée via une recherche du plus proche voisin par index spatial.

> Les deux voies ne sont **pas** équivalentes. Le repli géométrique est une commodité pour des données dépourvues d'identifiants explicites ; il peut rattacher un point à la mauvaise maille près des frontières du maillage ou là où les mailles sont petites. Chaque fois que des résultats faisant autorité importent, fournissez `ID_ABS` sur les observations et appuyez-vous sur la voie par identifiant — la recherche de la maille la plus proche doit être considérée comme un dernier recours, et non comme le comportement par défaut.

6.2 Le compartiment HYD : `ID_ABS` contre `ID_GIS`

Pour chaque compartiment *sauf un*, une maille conserve le **même identifiant** des deux côtés de l'outil : l'ID utilisé à l'intérieur de CaWaQS est l'ID utilisé à l'intérieur de CaWaQS-Viz. Cela permet à la majeure partie du code de faire circuler les ID à travers la couture CaWaQS / CaWaQS-Viz sans aucune conversion.

Le **compartiment HYD (hydrologique/rivière) est la seule exception.** Ses mailles sont identifiées différemment de chaque côté :

- à l'intérieur de CaWaQS, une maille HYD est identifiée par son `ID_ABS` (l'index absolu/interne CaWaQS) ;
- à l'intérieur de CaWaQS-Viz (le monde SIG), la même maille est identifiée par son `ID_GIS`.

Les deux sont reliés par une **table de correspondance HYD** produite avec les sorties de simulation, qui apparie chaque `ID_GIS` avec son `ID_ABS`. La traduction est appliquée **dans les deux sens**, aux deux extrémités du chemin de données :

- **En entrée** (lecture d'une observation HYD / alimentation de CaWaQS-Viz) : l'`ID_GIS` entrant côté CaWaQS-Viz est recherché dans la table de correspondance et **traduit en** `ID_ABS` afin que les résultats CaWaQS puissent être adressés.
- **En sortie** (report des résultats HYD vers le SIG) : la traduction inverse est appliquée, faisant correspondre `ID_ABS` à `ID_GIS`.



Conséquence pratique : si vous déboguez un résultat HYD et que les ID « ne collent pas », vérifiez si vous comparez un `ID_ABS` (côté CaWaQS) avec un `ID_GIS` (côté CaWaQS-Viz). Pour tout autre compartiment, les identifiants coïncident et aucune recherche de ce type n'a lieu ; seul HYD porte — et requiert — cette table de correspondance.

CHAPITRE 7

Architecture

Sommaire

7.1 Architecture séparée : HTAS et CWV 51
7.2 Les trois couches du backend 51
7.3 Exemple de flux de travail : le Budget Bar Plot 54
7.4 Portes d'entrée 55
7.4.1 Porte client — L1 - HT CLIENT - MACRO 55
7.4.2 Porte développeur — L2 - HT DEVELOPER - MICRO 55

Ce chapitre décrit l'architecture conceptuelle — la séparation HTAS/CWV et les trois couches du backend — ainsi que les **règles** qu'un développeur doit suivre en travaillant au sein de cette structure, et les deux portes d'entrée par lesquelles doivent passer tous les appels frontend → backend. Les recettes concrètes d'extension de l'application et l'outillage associé suivent dans les chapitres suivants.

7.1 Architecture séparée : HTAS et CWV

CaWaQS-Viz est construit au-dessus d'un backend scientifique autonome qu'il consomme au travers d'une interface publique bien définie. Comprendre le code revient donc à comprendre deux choses : comment l'application QGIS et le backend sont maintenus *séparés*, et comment le backend est organisé en interne en *trois couches*. Cette section décrit les deux, et énonce les règles strictes qu'un développeur doit suivre en travaillant au sein de cette structure.

CaWaQS-Viz est maintenu sous forme de deux paquets gardés **strictement indépendants** :

- `HydrologicalTwinAlphaSeries` **(HTAS)** — le backend scientifique autonome : traitement des simulations, structures du domaine (compartiments, maillages, observations), modèles de configuration, et la façade `HydrologicalTwin`. Il réside sous `external/HydrologicalTwinAlphaSeries`; dans la configuration publiée, ce chemin est un sous-module GitLab pointant vers le dépôt du backend autonome.
- `cawaqsviz` **(CWV)** — l'application QGIS autonome : boîtes de dialogue, flux de visualisation, intégration SIG, et adaptateurs côté application.

L'indépendance est un **invariant** architectural strict, et non une préférence de style :

- HTAS ne doit **jamais** importer `qgis.*`, `PyQt5`, `processing`, ni quoi que ce soit depuis `cawaqsviz/`. Si vous vous apprêtez à ajouter un tel import à l'intérieur de `external/HydrologicalTwinAlphaSeries/`, arrêtez — le travail relève du côté CWV;
- CaWaQS-Viz ne doit consommer HTAS qu'à travers son API publique — les deux portes d'entrée (Sec. 7.4) — et garder le code d'adaptation propre à QGIS côté application. Les imports HTAS à l'exécution relèvent de `interface/`; les modules `frontend/` n'accèdent au comportement du backend qu'à travers la passerelle de l'application;
- le backend est conçu pour être **utilisable depuis un notebook aujourd'hui et un serveur demain** (Sec. 4.1). Tout ce qui casse l'usage depuis un notebook en Python pur casse aussi le futur serveur;
- la synchronisation entre les deux dépôts se fait par le contrôle de version et l'épinglage des dépendances, non par duplication de code.

7.2 Les trois couches du backend

HTAS est organisé selon une **architecture à trois niveaux** dans laquelle chaque niveau peut appeler le niveau inférieur, jamais le niveau supérieur. Chaque niveau possède un module d'entrée canonique, et les imports pointent **strictement vers le bas** (L1 → L2 → L3) : jamais vers le haut, jamais latéralement au sein d'un niveau. Comme chaque dépendance pointe vers le bas, les couches ne peuvent jamais former une boucle d'import — le graphe de dépendances est acyclique.

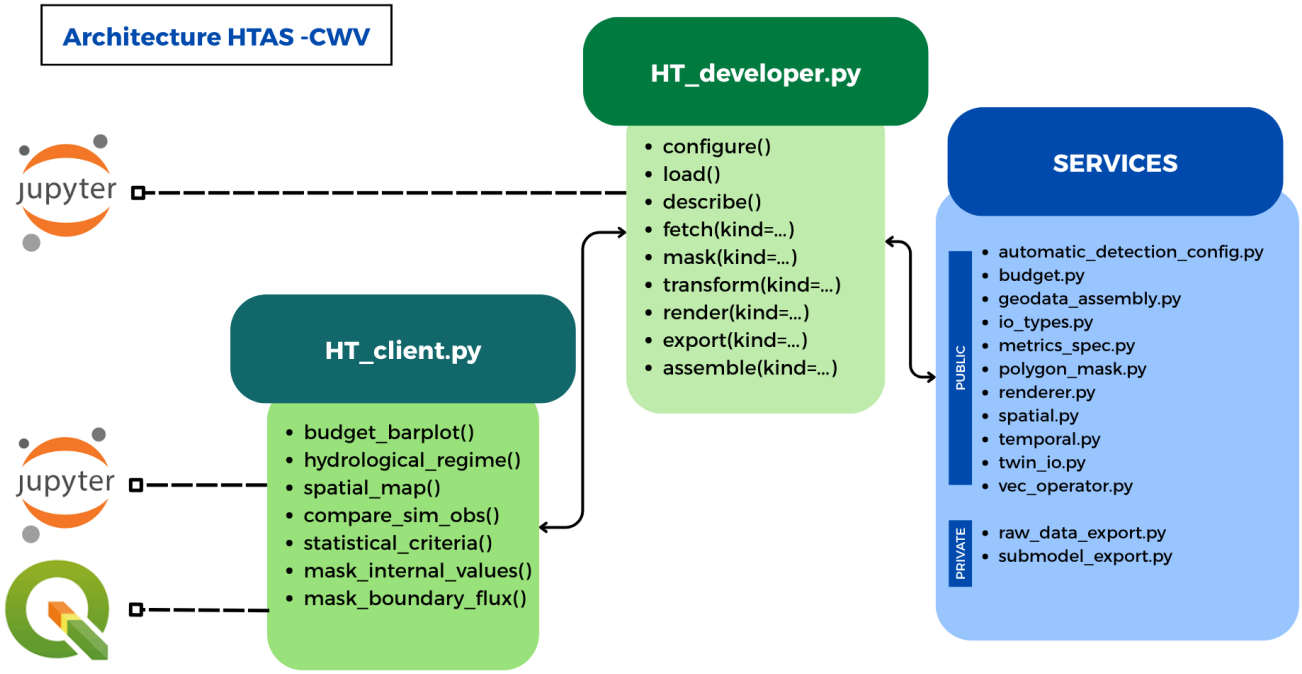


Figure 7.1 – Architecture CaWaQS-Viz + HTAS. Les trois couches du backend sont L1 (`HT_Client`), L2 (`HT_developer`), et L3 (`Services`).

```
L1 - HT CLIENT - MACRO        ht/client/hydrological_twin_client.py       (peut importer
    L2, L3)
L2 - HT DEVELOPER - MICRO     ht/developer/hydrological_twin_developer.py (peut importer
    L3)
L3 - SERVICES - ELEMENTARIES  services/                                   (n'importe rien
    dans ht/)
```

Code 7.1 – Les trois couches du backend et leurs modules d'entrée canoniques.

Avec les couches côté QGIS (frontend et interface), les rôles sont résumés dans le Tab. 7.1.

Couche	Rôle
Frontend `frontend/`	Boîtes de dialogue QGIS. Elles gèrent l'interaction utilisateur et acheminent un unique appel grossier par flux vers l'interface. `Qt/PyQt5` vit ici et seulement ici.
Interface `interface/`	`ExploreData` est la **seule** passerelle vers le backend ; `GeoDataProvider` transforme les couches vectorielles QGIS en `GeoDataFrame` pour que le backend reste agnostique de QGIS.
L1 - HT CLIENT - MACRO `ht/client/`	`HydrologicalTwinClient` expose une méthode par opération de dialogue orientée utilisateur. Chacune délègue à un `operations_client.run_*` correspondant qui possède toute la chaîne `fetch` → `transform` → `render` et renvoie une dataclass typée `*Result`. Zéro import `qgis/PyQt5` — utilisable depuis un notebook aujourd'hui, depuis un serveur HTTP demain.
L2 - HT DEVELOPER - MICRO `ht/developer/`	`HydrologicalTwin` est la façade micro-verbes exposant 8 verbes canoniques avec une machine à états explicite `EMPTY` → `CONFIGURED` → `LOADED` → `READY`. Utile pour l'exploration en notebook et pour les opérations pas encore promues au client.
L3 - SERVICES - ELEMENTARIES `services/`	Calculs et E/S les plus fins : opérateurs, moteur de rendu, lectures d'état du jumeau et provisionnement du cache disque (`twin_io.py`), et DTO de données feuilles (`io_types.py`). Aucun module sous `services/` n'importe depuis `ht/` — le graphe d'import est un DAG strictement descendant et acyclique.

Tableau 7.1 – Rôles du frontend, de l'interface et des trois couches du backend.

`ExploreData` appelle **le client** pour les opérations grossières (la voie normale) et peut encore atteindre le jumeau développeur *via* `client._twin` pour des appels micro-verbes de bas niveau. Une vue plus annotée du flux de données est donnée dans le Fig. 7.1.

Règle d'or — les imports pointent uniquement vers le bas. Jamais vers le haut, jamais latéralement au sein d'un niveau. Aucun module sous `services/` ne peut importer depuis `ht/client/` ou `ht/developer/`.

Les DTO de données partagés vivent à la feuille L3 (`services/public/io_types.py`) et sont ré-exportés par le `api_types.py` de L2, de sorte que l'arête reste descendante. Comme chaque dépendance pointe vers le bas, le graphe ne peut jamais former de boucle d'import — c'est un DAG strictement descendant et acyclique.

Règle d'or (corollaire) — L1 ne fait qu'orchestrer, il ne manipule pas les données. L1 (`hydrological_twin_client.p` et, derrière lui, `operations_client.py`) est de l'*orchestration pure* : il ne peut qu'*appeler* des micro-verbes L2 et *regrouper* leurs résultats typés dans une dataclass `*Result`. Il ne doit contenir **aucune manipulation de données propre** — pas de construction de `DataFrame`, pas de remise en forme ou d'agrégation de tableaux, pas de mise en forme d'unités/chemins/provenance, pas d'E/S disque, pas de calcul du domaine. Toute logique de ce type relève d'un cran plus *bas*, dans un micro-verbe L2 (`fetch`/`transform`/`render`/`export`, ou un nouveau `kind=`) ou dans L3 `services/`. Le test : si une ligne dans `ht/client/` fait autre chose qu'appeler une couche inférieure ou empaqueter/désempaqueter un résultat typé, elle est dans la mauvaise couche.



Le code assume d'être en transition : la flèche propre L2 → L3 unique décrite ici est la *cible*, pas encore le code littéral. Deux réserves sur l'état actuel qu'un développeur doit garder à l'esprit : (i) au sein de L2, `ht/developer/dispatch.py` et `ht/developer/handlers.py` sont des modules **transitoires** internes à L2 — aujourd'hui les micro-verbes routent `hydrological_twin_developer.py` → `dispatch.py` → `handlers.py` avant d'atteindre L3, et ces deux modules sont destinés à être fondus dans les corps des micro-verbes L2 (voir `external/HydrologicalTwinAlphaSeries/docs/TODO.md`) ; (ii) quelques imports descendants entre couches (par ex. la réutilisation de la fonction de récupération de la matrice de simulation à l'intérieur de plusieurs méthodes L2) sont tolérés pour l'instant et seront resserrés plus tard — ils ne posent actuellement pas de problème pour l'usage en notebook.

7.3 Exemple de flux de travail : le Budget Bar Plot

Le déroulé suivant montre ce qui se passe, pas à pas, lorsqu'un utilisateur crée un histogramme de bilan hydrique. C'est l'un des flux les plus complets, enchaînant `fetch` → `transform` → `render`, et il est utile pour comprendre le nombre d'allers-retours entre couches — en particulier lors de la conception de la future API serveur.

1. Dans la boîte de dialogue `DialogBudgetbarplot`, l'utilisateur sélectionne des variables, une période, une fréquence et une agrégation.
2. La boîte de dialogue émet un *unique* appel grossier vers `ExploreData`, qui le transmet à `HydrologicalTwinClient` lequel appelle à son tour `operations_client.run_budget_barplot`.
3. Pour chaque variable (pluie, ETR, infiltration, ruissellement), l'orchestration appelle `twin.fetch` (lit la matrice de simulation depuis le disque à travers la couche services) puis `twin.transform` (moyenne spatiale + agrégation temporelle).
4. Un dernier appel `twin.render` produit le tracé ; le moteur de rendu écrit un PNG et un CSV sur le disque et renvoie leurs chemins.

5. Les chemins sont regroupés dans une dataclass `BudgetBarplotResult` et remontés le long de la chaîne jusqu'à la boîte de dialogue, qui ouvre le fichier du tracé.

Ce que cela révèle pour la conception de l'API :

- aujourd'hui, tout ce qui se trouve entre `ExploreData` et le jumeau développeur est **en processus**. La couture réseau naturelle est la frontière `ExploreData` → `Client`, pas `ExploreData` → `HydrologicalTwin` ;
- **une opération grossière = un aller-retour serveur**. La boîte de dialogue appelle `client.budget_barplot(...)` une fois ; la boucle interne de 4 fetches + 4 transforms + 1 render reste côté serveur à l'intérieur de `run_budget_barplot`. L'ancien coût de « 9 appels par tracé » s'effondre en un seul appel à la frontière réseau ;
- **le client *est* la future surface HTTP.** Chaque méthode de `HydrologicalTwinClient` correspond proprement à un point de terminaison, avec une dataclass typée `*Result` comme schéma de réponse. L'API micro-verbes développeur reste disponible pour l'exploration façon notebook ;
- **le rendu écrit encore sur le disque.** Lorsque le client passera côté serveur, les sorties de rendu devront renvoyer des données (ou des URL signées) plutôt que des chemins de fichiers locaux — les dataclass `*Result` sont le bon endroit pour faire évoluer ce contrat.

7.4 Portes d'entrée

Tous les appels frontend → backend passent par l'une de deux surfaces. Ne les contournez pas.

7.4.1 Porte client — L1 - HT CLIENT - MACRO

`HydrologicalTwinClient` (`ht/client/hydrological_twin_client.py`) expose **une méthode par opération de dialogue orientée utilisateur** :

- `budget_barplot`, `hydrological_regime` ;
- `spatial_map_watbal`, `spatial_map_aq` ;
- `compare_sim_obs`, `statistical_criteria` ;
- `mask_watbal`, `mask_hyd_boundary`, `mask_aq_boundary`.

Plus les méthodes de cycle de vie `configure`, `load`, `describe`, `is_ready`, et la classmethod « one-shot » `build`. Chaque opération enveloppe une chaîne complète `fetch` → `transform` → `render` côté serveur et renvoie une dataclass typée `*Result` (définie dans `ht/client/api_types.py`, orchestrée par `ht/client/operations_client.py`). **C'est la surface qu'exposera un futur serveur HTTP :** une opération grossière = un aller-retour. Les boîtes de dialogue frontend devraient privilégier cette porte.

7.4.2 Porte développeur — L2 - HT DEVELOPER - MICRO

`HydrologicalTwin` (`ht/developer/hydrological_twin_developer.py`) expose les 8 verbes canoniques :

- cycle de vie (en ligne) : `configure`, `load`, `describe`, `export` ;
- dispatch (routé par `kind=` à travers les modules transitoires `dispatch.py` → `handlers.py`, puis vers L3) : `fetch`, `mask`, `transform`, `render`.

Machine à états : `EMPTY` → `CONFIGURED` → `LOADED` → `READY`. Chaque verbe vérifie l'état via `_require_state` et lève `InvalidStateError` s'il est appelé trop tôt. La porte développeur est la surface d'**utilisateur avancé /**

notebook / exploration. Utilisez-la pour les opérations pas encore promues au client, et pour le travail de bas niveau ; `ExploreData` peut encore l'atteindre *via* `client._twin` au besoin.

CHAPITRE 8

Flux de développement

Sommaire

8.1 Ajouter une nouvelle opération de dialogue 57
8.2 Lors d'une modification — liste de contrôle 57
8.3 Conventions 58
8.4 Mise à jour de version 58
8.5 Documentation du code 59

Ce chapitre rassemble les **règles et processus** qu'un développeur doit suivre en travaillant sur CaWaQS-Viz : comment étendre l'application avec une nouvelle opération, la liste de contrôle à parcourir avant de committer, les conventions du projet, et comment le versionnage et la documentation du code sont gérés.

8.1 Ajouter une nouvelle opération de dialogue

Pour ajouter un nouveau flux au niveau dialogue, suivez trois étapes :

1. définissez une dataclass `*Result` dans `ht/client/api_types.py` ;
2. implémentez `run_<name>` dans `ht/client/operations_client.py` — il possède l'orchestration `fetch` → `transf`
3. ajoutez une méthode de façade d'une ligne sur `HydrologicalTwinClient`, puis appelez-la depuis `ExploreData`.

Les corps de méthode sur la classe client sont volontairement minces (≤ 5 instructions). Gardez-les ainsi.

8.2 Lors d'une modification — liste de contrôle

- vous touchez au code d'une boîte de dialogue ? → `frontend/Dialog*.py`. Passez par `ExploreData` ;
- vous ajoutez un nouveau flux au niveau dialogue ? → la recette en trois étapes ci-dessus (Sec. 8.1) ;
- vous ajoutez une nouvelle primitive de calcul ? → `ht/developer/handlers.py` (calcul) ou `services/public/twi` (lectures d'état L3 / E/S disque) ; routez *via* `dispatch.py` avec un nouveau `kind=` ;

- vous avez trouvé un import `qgis` / `PyQt5` dans `external/HydrologicalTwinAlphaSeries/` ? → bug. Déplacez-le vers `interface/` ou `frontend/` ;
- vous avez trouvé un module sous `services/` qui importe depuis `ht/client/` ou `ht/developer/` ? → bug. C'est un import *ascendant* ; les imports doivent pointer uniquement vers le bas ;
- vous avez trouvé un module frontend qui contourne `ExploreData` pour atteindre les internes de HTAS ? → bug. Passez par la porte client ;
- vous avez trouvé une manipulation de données dans `ht/client/` (construction de `DataFrame`, remise en forme/agrégation de tableaux, mise en forme d'unités/chemins, E/S disque, maths du domaine) ? → bug. L1 ne fait qu'orchestrer.

8.3 Conventions

- **Environnement :** géré par `pixi` (`pixi.toml`, `pixi.lock`). QGIS, Python et toutes les dépendances sont figés dans un unique environnement reproductible (Sec. 1.1) ;
- **Aides à la synchronisation :** `./verifState.sh` (diagnostics) et `./updateProject.sh` (synchronisation ordonnée multi-dépôts / sous-modules) ;
- **Découverte pré-cycle-de-vie** (sans jumeau requis) : `HydrologicalTwinClient.detect_from_out_caw(...)` et `detect_project_neighbors(...)` — aides statiques utilisées par `DialogSettings` pour pré-remplir les chemins avant `configure` / `load` ;
- **Cale de compatibilité :** `cawaqsviz/backend`, si présent, ré-exporte les symboles HTAS. L'implémentation réelle ne vit que dans `external/HydrologicalTwinAlphaSeries/`.

8.4 Mise à jour de version

La version du plug-in est spécifiée dans le fichier `metadata.txt` figurant dans l'extrait de code 8.1.

```
[general]
name=CaWaQS-Viz
qgisMinimumVersion=3.0
description=Description
version=0.1.16
author=Lise-Marie GIROD (Mines Paris - PSL), Nicolas Flipo (Mines Paris - PSL)
email=lise-marie.girod@minesparis.psl.eu

about=Python QGIS-based vizualization software for CaWaQS3.x

repository=https://gitlab.com/gviz/cawaqsviz
```

Code 8.1 – Informations obligatoires à renseigner dans le fichier `metadata.txt`

8.5 Documentation du code

La documentation du code est publiée à l'adresse suivante via GitLab Pages : https://cawaqsviz-gviz-adf347ff6d9d45c174db091497a2dfc0f14305cdb4ed9070.gitlab.io/. La documentation est maintenue à travers les docstrings des modules de l'application, en suivant le formatage montré dans l'exemple ci-dessous.

```python
class Exemple:
    """
    Resume des fonctionnalites de la classe.
    """

    def __init__(self, arg1):
        """
        Methode constructeur.

        :param arg1: descriptif du parametre
        :type arg1: str
        """
        self.arg1 = arg1

    def do_something(self, arg1: list) -> str:
        """
        Descriptif de la fonction.

        :param arg1: descriptif du parametre
        :type arg1: list
        :return: descriptif de ce que la fonction retourne
        :rtype: str
        """
        return ", ".join(map(str, arg1))
```

Code 8.2 – Formatage des docstrings pour l'intégration automatique dans la documentation du code

La compilation est réalisée par **`Sphinx`**[1]. Le répertoire **`docs/sources`** contient les fichiers de configuration (**`Conf.py`**), ainsi que le contenu de la documentation. Lors de l'intégration de nouveaux modules dans le plug-in, leur ajout à l'index (**`index.rst`**) est nécessaire pour que la docstring du code apparaisse sur la page de documentation. Cet index intègre 3 fichiers : **`frontend.rst`**, **`interface.rst`**, **`backend.rst`**, qui contiennent les classes à documenter. L'intégration d'un nouveau module dans l'application requiert son ajout à ces fichiers au format de l'extrait de code 8.3. Les mises à jour des fichiers sources de la documentation doivent être committées et synchronisées avec les dépôts.

```
Module
------

.. automodule:: cawaqsviz.Module
   :members:
   :show-inheritance:
```

1. dont la documentation est accessible via https://www.sphinx-doc.org/en/master/

```
   :undoc-members:
```

Code 8.3 – Formatage des docstrings pour l'intégration automatique dans la documentation du code

Enfin, à chaque nouveau commit sur les branches `Main` ou `Beta`, la documentation est compilée selon le contenu du répertoire `docs/source` du dépôt.

```
image: python:3.9

stages:
  - build
  - deploy

before_script:
  - pip install -U pip
  - pip install -r requirements.txt

build-docs:
  stage: build
  script:
    - sphinx-build -b html docs/source public
  artifacts:
    paths:
      - public

pages:
  stage: deploy
  script:
    - echo "Pages deployed"
  artifacts:
    paths:
      - public
  only:
    - main
    - beta
```

CHAPITRE 9

Outillage du développeur

Sommaire

9.1 Outils de débogage 62
9.1.1 L'extension `Plugin Reloader` 62
9.1.2 L'extension QGIS `First Aid` 62
9.1.3 VS-Code 63
9.2 Créer des boîtes de dialogue avec `Qt Designer` 67

Ce chapitre couvre les **outils optionnels** qui facilitent le développement au quotidien : les utilitaires de débogage (rechargement du plug-in, inspection des erreurs, et un débogueur VS-Code complet attaché à QGIS), et `Qt Designer` pour la création des boîtes de dialogue.

9.1 Outils de débogage

9.1.1 L'extension `Plugin Reloader`

Cet utilitaire permet de recharger une extension après une modification de code. Il n'est ainsi pas nécessaire de relancer QGIS après chaque changement de code.

L'extension peut être installée depuis le gestionnaire d'extensions de `QGIS`.



Figure 9.1 – Fiche descriptive de `Plugin Reloader`

9.1.2 L'extension QGIS `First Aid`

L'extension `First Aid` permet d'investiguer les erreurs survenant lors de l'exécution de l'extension.

9.1.3 VS-Code

L'extension **debugvs** permet de connecter une instance de débogage **VS-Code** à l'exécution de l'extension depuis QGIS. Cette configuration de débogage permet d'utiliser toutes les fonctionnalités de VS Code pendant le débogage (points d'arrêt, watch, pile d'appels...). La mise en place de cette configuration de débogage se fait comme suit :

1. installez **debugpy** avec la commande **!pip install debugpy** depuis l'invite de commande Python de QGIS ;
2. si le répertoire de développement de l'extension n'est pas situé dans `C://Users/`*USER*`/AppData//QGIS/QGIS3/profiles/default/python`, créez un lien symbolique dans ce répertoire pointant vers le répertoire de développement.
3. installez l'extension **debugvs** en téléchargeant son script depuis le dépôt GitLab https://github.com/lmotta/debug_vs et activez l'extension en important le dossier **.zip** dans **QGIS** (cf. Fig. 9.2) ;

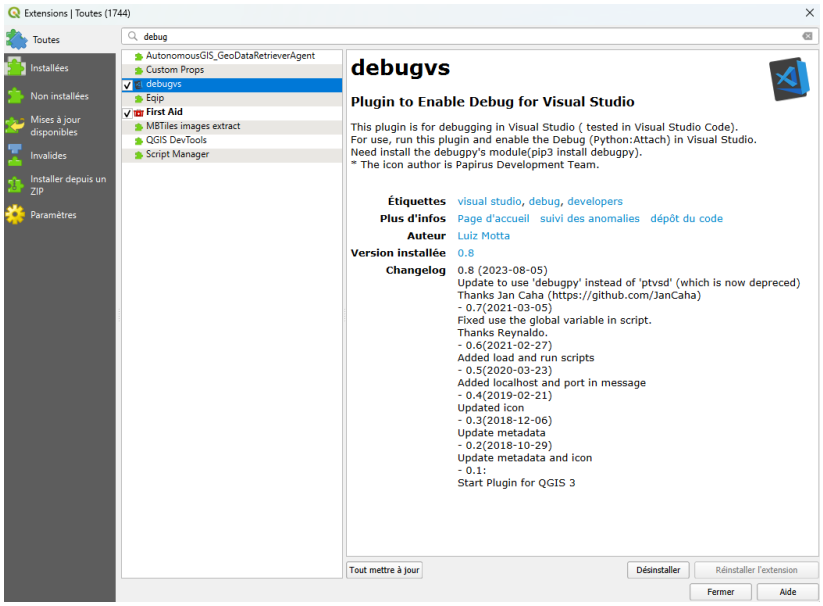


Figure 9.2 – Extension **debugvs**

4. configurez VS Code pour se connecter à QGIS pendant une session de débogage :
   (a) dans le répertoire de développement CaWaQS-Viz, créez un sous-répertoire **.vscode** (ajoutez ce répertoire au fichier **.gitignore**) ;
   (b) créez un fichier **settings.json** dans le sous-répertoire **.vscode** et copiez le contenu du code 9.1. Modifiez éventuellement les chemins des répertoires pour correspondre à votre installation QGIS et Python et à votre système d'exploitation. Celui-ci est pour Windows.

```
1  {
2
3      "workbench.colorCustomizations": {
```

```
4                 "titleBar.activeBackground": "#006823",
5                 "editor.background": "#000000"
6             },
7
8             "python.linting.pylintEnabled": true,
9             "python.jediEnabled": true,
10
11
12            "python.pythonPath": "C:/OSGeo4W64/bin/python-qgis.bat",
13
14
15            "python.linting.pylintArgs": [
16                "--extension-pkg-whitelist=PyQt5,qgis,db_manager",
17                "--disable=all",
18                "--enable=F,E,unreachable,duplicate-key,unnecessary-semicolon,
    global-variable-not-assigned,unused-variable,binary-op-exception,bad-format-
    string,anomalous-backslash-in-string,bad-open-mode"
19            ],
20
21
22            "terminal.integrated.env.windows": {
23
24                "PATH": "C:\\OSGeo4W\\apps\\qgis\\bin;C:\\OSGeo4W\\apps\\
    Python312;C:\\OSGeo4W\\apps\\Python312\\Scripts;C:\\OSGeo4W\\apps\\Qt5\\bin;C
    :\\OSGeo4W\\apps\\Python312\\Scripts;C:\\OSGeo4W\\bin;C:\\Windows\\System32;C
    :\\Windows;C:\\Windows\\system32\\WBem;C:\\OSGeo4W\\apps\\Python312\\lib\\
    site-packages\\pywin32_system32;C:\\OSGeo4W\\apps\\Python312\\lib\\site-
    packages\\numpy\\.libs",
25
26                "PYTHONHOME": "C:\\OSGeo4W\\apps\\Python312",
27                "PYTHONPATH": "C:\\OSGeo4W\\apps\\qgis\\python;%PYTHONPATH%",
28
29                "GDAL_DATA": "C:\\OSGeo4W\\share\\gdal",
30                "GDAL_DRIVER_PATH": "C:\\OSGeo4W\\bin\\gdalplugins",
31                "GDAL_FILENAME_IS_UTF8": "YES",
32
33                "GEOTIFF_CSV": "C:\\OSGeo4W\\share\\epsg_csv",
34
35                "O4W_QT_BINARIES": "C:/OSGeo4W/apps/Qt5/bin",
36                "O4W_QT_DOC": "C:/OSGeo4W/apps/Qt5/doc",
37                "O4W_QT_HEADERS": "C:/OSGeo4W/apps/Qt5/include",
38                "O4W_QT_LIBRARIES": "C:/OSGeo4W/apps/Qt5/lib",
39                "O4W_QT_PLUGINS": "C:/OSGeo4W/apps/Qt5/plugins",
40                "O4W_QT_PREFIX": "C:/OSGeo4W/apps/Qt5",
41                "O4W_QT_TRANSLATIONS": "C:/OSGeo4W/apps/Qt5/translations",
42                "QT_PLUGIN_PATH": "C:\\OSGeo4W\\apps\\qgis\\qtplugins;C:\\OSGeo4W
    \\apps\\Qt5\\plugins",
43
44                "QGIS_PREFIX_PATH": "C:/OSGeo4W/apps/qgis",
45
46                "VSI_CACHE": "TRUE",
```

```
                "VSI_CACHE_SIZE": "1000000"
            },
            "python.languageServer": "Jedi",
        }
```

Code 9.1 – Fichier de configuration du workspace. Ce fichier définit la configuration du workspace Visual Studio Code pour le développement d'extensions QGIS. Il adapte l'environnement Python de VS Code à celui utilisé par QGIS (installé via OSGeo4W).

(c) créez un fichier **`launch.json`** dans le sous-répertoire **`.vscode`**, copiez le contenu du code 9.2, modifiez les chemins **`localRoot`** et **`remoteRoot`** pour correspondre à la configuration de votre workspace.

```
{
    "version": "0.2.0",
    "configurations": [
        {
            "name": "Attach QGIS Debug",
            "type": "python",
            "request": "attach",
            "connect": {
                "host": "localhost",
                "port": 5678
            },
            "pathMappings": [
                {
                    "localRoot": "C:\\Program Files\\$QGIS$\\apps\\qgis-ltr\\
    python\\plugins\\cawaqsviz",
                    "remoteRoot": "C:\\Users\\$USER$\\AppData\\Roaming\\QGIS\\
    QGIS3\\profiles\\default\\python\\cawaqsviz"
                }
            ],
            "justMyCode": false
        }
    ]
}
```

Code 9.2 – Fichier de configuration du débogueur VS-Code. Dans cette configuration, la section « connect » indique l'adresse et le port sur lesquels QGIS expose la session de débogage (ici localhost:5678). Le bloc « pathMappings » établit la correspondance entre les chemins locaux du projet ouverts dans VS Code (localRoot) et les chemins utilisés par QGIS pour exécuter l'extension (remoteRoot). Cette correspondance est essentielle pour que les points d'arrêt placés dans VS Code soient correctement associés aux fichiers réellement exécutés par QGIS.

5. Établissez la connexion QGIS/VS-Code

   (a) autorisez la connexion du débogueur à QGIS via l'extension **`debugvs`** (cf. Fig. 9.3) ;

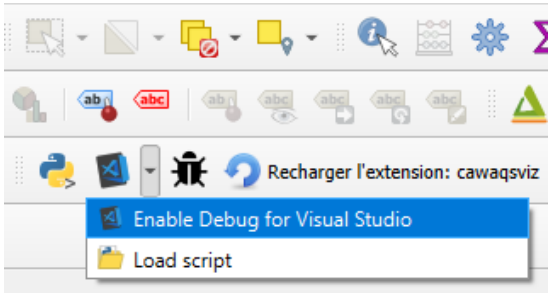


Figure 9.3 – Établissement d'une connexion QGIS/VS-Code depuis l'extension `debugvs`

(b) lancez la session de débogage VS-Code (cf. Fig. 9.4).

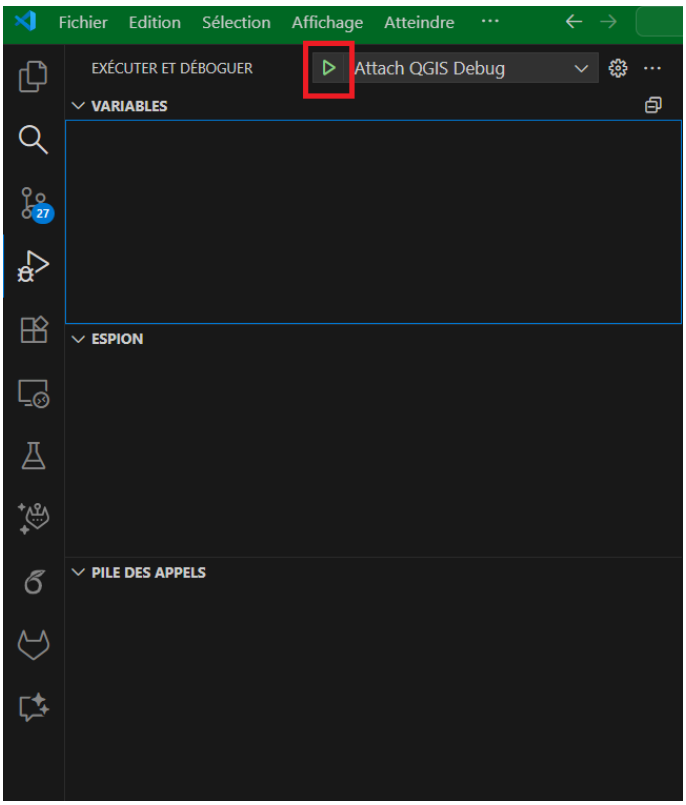


Figure 9.4 – Lancement d'une session de débogage depuis VS-Code

La connexion entre le débogueur et QGIS est terminée depuis VS-Code en cliquant sur `"Disconnected"` dans la barre d'outils de débogage (cf. Fig. 9.5)



Figure 9.5 – Barre d'outils du débogueur VS-Code

9.2 Créer des boîtes de dialogue avec Qt Designer

L'application **Qt Designer**, fournie avec l'installation de QGIS (installation avec OSGEO), est un outil de création de boîtes de dialogue sous forme de fichiers .ui. Dans l'application CaWaQS-Viz, toutes les boîtes de dialogue disposent de leur propre fichier **.py** auquel le fichier **.ui**, de même nom, est associé à la classe Python de la boîte de dialogue.

```python
# This loads your .ui file so that PyQt can populate your plugin with the elements from Qt Designer
FORM_CLASS, _ = uic.loadUiType(
    os.path.join(os.path.dirname(__file__), "DialogComparesimobs.ui")
)


class DialogComparesimobs(Dialog, FORM_CLASS):
    def __init__(self, iface:QgisInterface, log_file_path:str, parent=None):
        """Constructor Method

        :param iface: Qgis Interface
        :type iface: QgisInterface
        :param logFilePath: Path of the cawaqs-viz log file, defaults to None
        :type logFilePath: str, optional
        :param parent: Parent class, defaults to None
        :type parent: _type_, optional
        """
        super(DialogComparesimobs, self).__init__(parent)
        self.setupUi(self)
        self.iface = iface
        self.logFilePath = log_file_path
        self.simData = False
```

Code 9.3 – Initialisation de la boîte de dialogue "Compare SIM/OBS" depuis un fichier .ui

Bibliographie

GUPTA, H. V., CLARK, M. P., VRUGT, J. A., ABRAMOWITZ, G. et YE, M. (2012). Towards a comprehensive assessment of model structural adequacy. *Water Resources Research*, 48(8). _eprint : https ://agupubs.onlinelibrary.wiley.com/doi/pdf/10.1029/2011WR011044.

NASH, J. E. et SUTCLIFFE, J. V. (1970). River flow forecasting through conceptual models part I — A discussion of principles. *Journal of Hydrology*, 10(3):282–290.

Table des figures

1 L'écosystème CaWaQS-Viz — position du plug-in au sein de la chaîne d'outils CaWaQS. . . . vi

1.1 Boîte de dialogue QGIS pour la gestion d'extensions. 6
1.2 Activation de l'extension CaWaQS-Viz dans l'application QGIS. 7

2.1 Boîte de dialogue d'accueil de l'application CaWaQS-Viz. 11
2.2 La boîte de dialogue "`Settings`" du plug-in, avec la création d'un nouveau projet sélectionnée. 12

3.1 Boîte de dialogue des outils de calage. 17
3.2 Boîte de dialogue de modification des fichiers de paramétrisation. 18
3.3 Boîte de dialogue de liaison d'une table attributaire à sa résolution. 19
3.4 Boîte de dialogue "`Statistical Criteria`". 22
3.5 Boîte de dialogue de calcul du coefficient de ruissellement. 23
3.6 Fenêtre de dialogue "`Compare sim/obs`". 24
3.7 Spatialisation du flux d'infiltration moyen interannuel [mm] (a) et de la piézométrie [$mNGF$] (b) simulés à l'échelle de l'hydrosystème du Loing 25
3.8 Exemple de sortie graphique du bilan hydrologique pluriannuel mensuel simulé 26
3.9 Boîte de dialogue "`Hydrological Regime`" pour le calcul des régimes hydrologiques pluriannuels mensuels. 27
3.10 Exemple de sortie graphique du régime hydrologique pluriannuel mensuel simulé 27
3.11 Boîte de dialogue "`Extract Data from a Sub Area`". 28
3.12 "`AQ boundary geopackage`". 31

5.1 Champs de configuration du compartiment WATBAL 41
5.2 Champs de configuration du compartiment HYD 41
5.3 Champs de configuration du compartiment NSAT 41
5.4 Champs de configuration du compartiment AQ 42

5.5 Fenêtre de sélection de l'identifiant du champ contenant l'"`ID ABS`" dans la table attributaire dans le cas d'identifiants de champs hétérogènes entre les couches 42
5.6 Champs de configuration des points d'extraction du compartiment "`HYD`" 43
5.7 Champs de configuration des points d'extraction du compartiment "`AQ`" 43

7.1 Architecture CaWaQS-Viz + HTAS. Les trois couches du backend sont L1 (`HT_Client`), L2 (`HT_developer`), et L3 (`Services`). 52

9.1 Fiche descriptive de `Plugin Reloader` 62
9.2 Extension `debugvs` 63
9.3 Établissement d'une connexion QGIS/VS-Code depuis l'extension `debugvs` 66
9.4 Lancement d'une session de débogage depuis VS-Code 66
9.5 Barre d'outils du débogueur VS-Code 66

Liste des tableaux

1 Dernières implémentations dans l'application vii

3.1 Fichiers de paramétrisation pris en charge par `CaWaQS-Viz`. 20
3.2 Noms des champs des tables de sortie pour chaque critère statistique 22
3.3 Tuples `(compartment, outtype, param, unit)` acceptés par **`mask_internal_values`**. . . . 32

5.1 Résolutions CaWaQS pouvant être importées comme résolution de compartiment. 39
5.2 Structure des couches SIG des points d'extraction et sorties graphiques associées 40

7.1 Rôles du frontend, de l'interface et des trois couches du backend. 53

Liste des codes

2.1 Organisation attendue du répertoire de sorties CaWaQS. 14
2.2 Organisation attendue autour du fichier de projet QGIS. 14
3.1 Exemple de `specs` pour `mask_internal_values`. 32
3.2 Exemple d'appel de `mask_aq_boundary`. 33
5.1 Contenu d'un fichier de configuration des géométries 44
7.1 Les trois couches du backend et leurs modules d'entrée canoniques. 51
8.1 Informations obligatoires à renseigner dans le fichier `metadata.txt` 58
8.2 Formatage des docstrings pour l'intégration automatique dans la documentation du code . . 59
8.3 Formatage des docstrings pour l'intégration automatique dans la documentation du code . . 59
9.1 Fichier de configuration du workspace. Ce fichier définit la configuration du workspace Visual Studio Code pour le développement d'extensions QGIS. Il adapte l'environnement Python de VS Code à celui utilisé par QGIS (installé via OSGeo4W). 63
9.2 Fichier de configuration du débogueur VS-Code. Dans cette configuration, la section « connect » indique l'adresse et le port sur lesquels QGIS expose la session de débogage (ici localhost :5678). Le bloc « pathMappings » établit la correspondance entre les chemins locaux du projet ouverts dans VS Code (localRoot) et les chemins utilisés par QGIS pour exécuter l'extension (remoteRoot). Cette correspondance est essentielle pour que les points d'arrêt placés dans VS Code soient correctement associés aux fichiers réellement exécutés par QGIS. 65
9.3 Initialisation de la boîte de dialogue "Compare SIM/OBS" depuis un fichier .ui 67